Python

30/3/26 Keywords :- Reserved words or predefined words. Keywords is a universal standard words which is predefined by the developers which can be access its functionalities but cannot be modify the original task.

* There are 35 keywords in Python.
* Syntax :-

```
import keywords
keyword.kwlist
```

There are 3 special keywords in Python:-

1. True
2. False
3. None

They can be stored as value to a variable has they contain some internal values.

Ex:-
```
a = True
b = False
C = None
print(a)  print(b)  print(C)
```
→ True → False → Empty

Variables :- It is a container which is used to store the value or name given to the memory location where the value has been stored.

Ex:- a = 42 , b = "Hello" , C = 12.43

Stack	heap
a	→ 10x11
10x11	42

id() :- It is used to access the actual memory address of the value.

* we can create multiple variable in a single line.

* Syntax :-

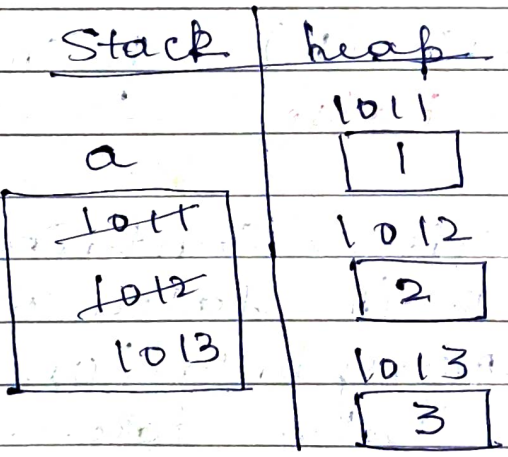
var1, var2, var3.... varn = val1, val2, val3 ... valn

Ex:- a = 10
id(a)
→ 140704691287448

a, b, c = 10, 20, 30
print(a) print(b)
→ 10 → 20

* Here number of values & variables must be same otherwise it throws error. Variables should not be same.

Ex:- a, a, a = 1, 2, 3
print(a)
→ 3



Identifiers :- It is a meaning full name given to user defined entities like variable, class, functions.

* Variables can be Identifiers but

Identifiers Cannot be a Variables.

Ex:- Age = 32
print(Age)
→ 32

name = 'Kavana'
print(name)
→ 'Kavana'

Rules for Identifiers :-

1. It should not be a keywords.
2. It should not start with digits
3. It should not Contain any special Character except underscore (_).
4. It should not Contain any white space
5. It Can have alphabet, alphanumeric & group of alphabets.

31|3|26 Copy operations :- Procas of Copying the Content from one Variable to another Variable is Called as Copy operations.

* There are three types of Copy operations:-
1. General Copy
2. Shallow Copy
3. Deep Copy

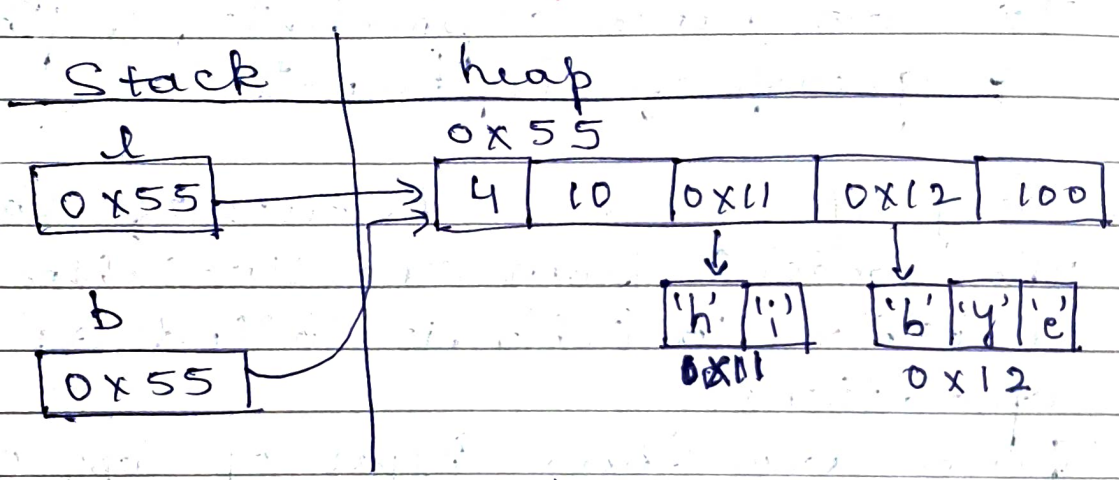
1. General Copy :- General Copy or reference Copy used to Copying the Content from one Variable to another Variable with the same Memory address.

Syntax :-

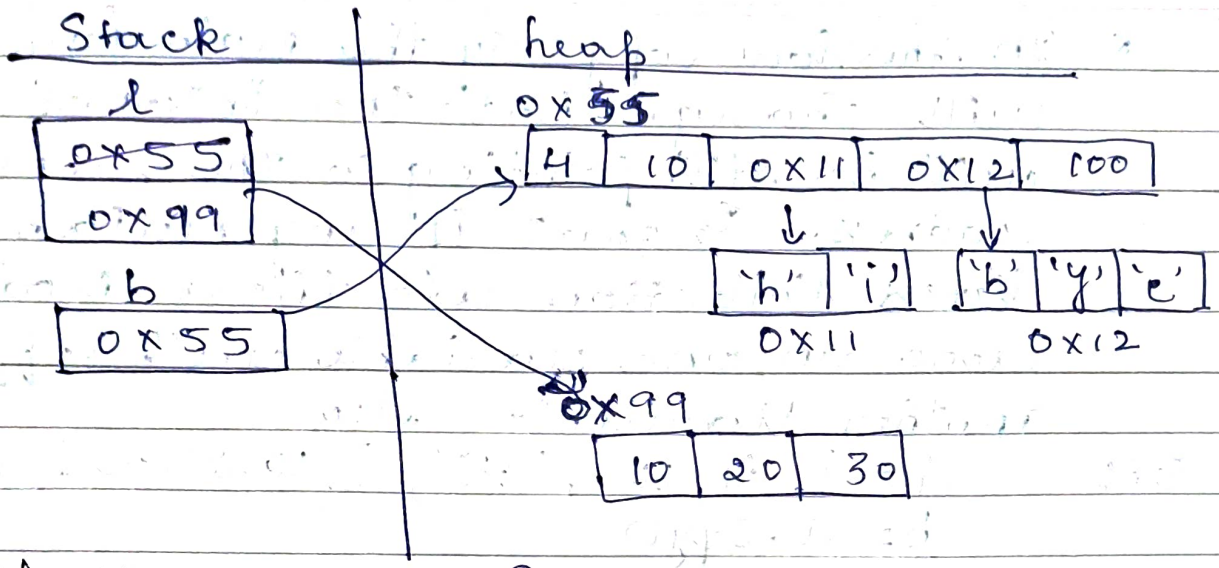
dest_var = source_var

* Here has the variables are pointing to same memory address, if data is of Mutable type modification done on one variable affects other variable as well.

Ex:- `l = [4, 10, 'hi', 'bye', 100]`
`b = l`
`l`
→ `[4, 10, 'hi', 'bye', 100]`
`b`
→ `[4, 10, 'hi', 'bye', 100]`



Ex:- `l = [4, 10, 'hi', 'bye', 100]`
`b = l`
`l.append(50)`
`print(l)`
→ `[4, 10, 'hi', 'bye', 100, 50]`
`print(b)`
→ `[4, 10, 'hi', 'bye', 100, 50]`
`l = [10, 20, 30]` (new initialize)
`print(b)`
→ `[4, 10, 'hi', 'bye', 100, 50]`
`print(l)`
→ `[10, 20, 30]`

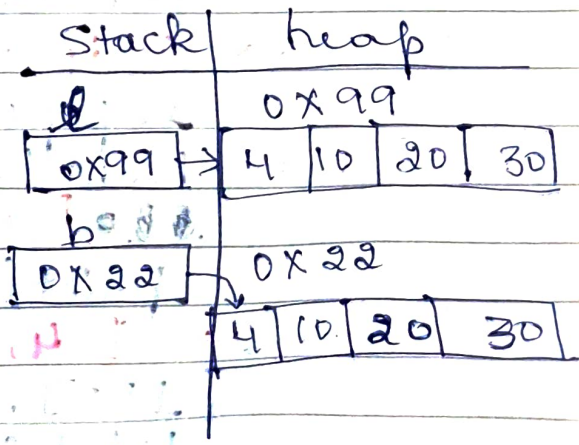


2. Shallow Copy :- Procus of Copying the Content from one variable to another variable with different memory address.

* It supports only mutable datatypes.
* Syntax :-

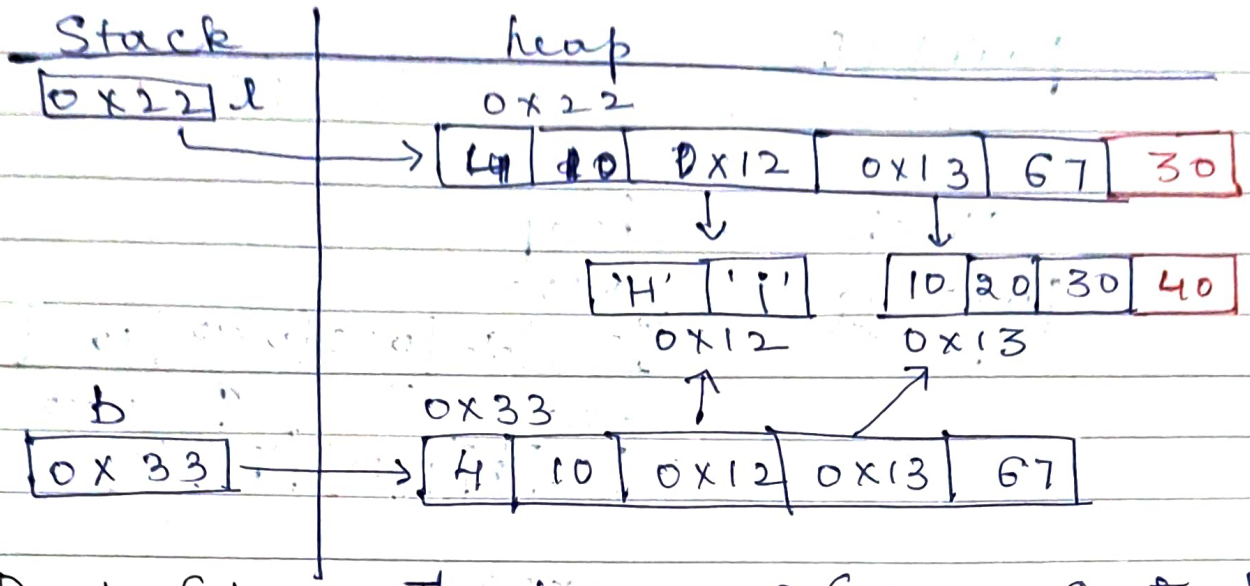
dest_var = source_var.copy().

Ex:-
```
l = [4, 10, 20, 30]
b = l.copy()
print(l)
→ [4, 10, 20, 30]
print(b)
→ [4, 10, 20, 30]
l.append(500)
print(l)
→ [4, 10, 20, 30, 500]
print(b)
→ [4, 10, 20, 30]
```



* In Shallow Copy only the main layers address will change but the internal values will be still pointing to the same address. So that if any mutable nested ~~listed~~ Collection are there Modification done on one variable will affect another variable also.

Ex:-
```
l = [4, 10, 'Hi', [10,20,30], 67]
b = l.copy()
print(l)
→ [4, 10, 'Hi', [10,20,30], 67]
print(b)
→ [4, 10, 'Hi', [10,20,30], 67]
l.append(30)
print(l)
→ [4, 10, 'Hi', [10,20,30], 67, 30]
print(b)
→ [4, 10, 'Hi', [10,20,30], 67]
l[3].append(40)
print(l)
→ [4, 10, 'Hi', [10,20,30,40], 67]
print(b)
→ [4, 10, 'Hi', [10,20,30,40], 67]
l.pop()
print(l)
→ [4, 10, 'Hi', [10,20,30,40]]
print(b)
→ [4, 10, 'Hi', [10,20,30,40], 67]
l[2].pop()
print(l)
→ [4, 10, [10,20,30,40]]
print(b)
→ [4, 10, 'Hi', [10,20,30,40], 67]
```



1/4/26

3. Deep Copy:- The process of Copying Content from one variable to another variable with the different Memory address including the nested mutable values.

* Syntax:-

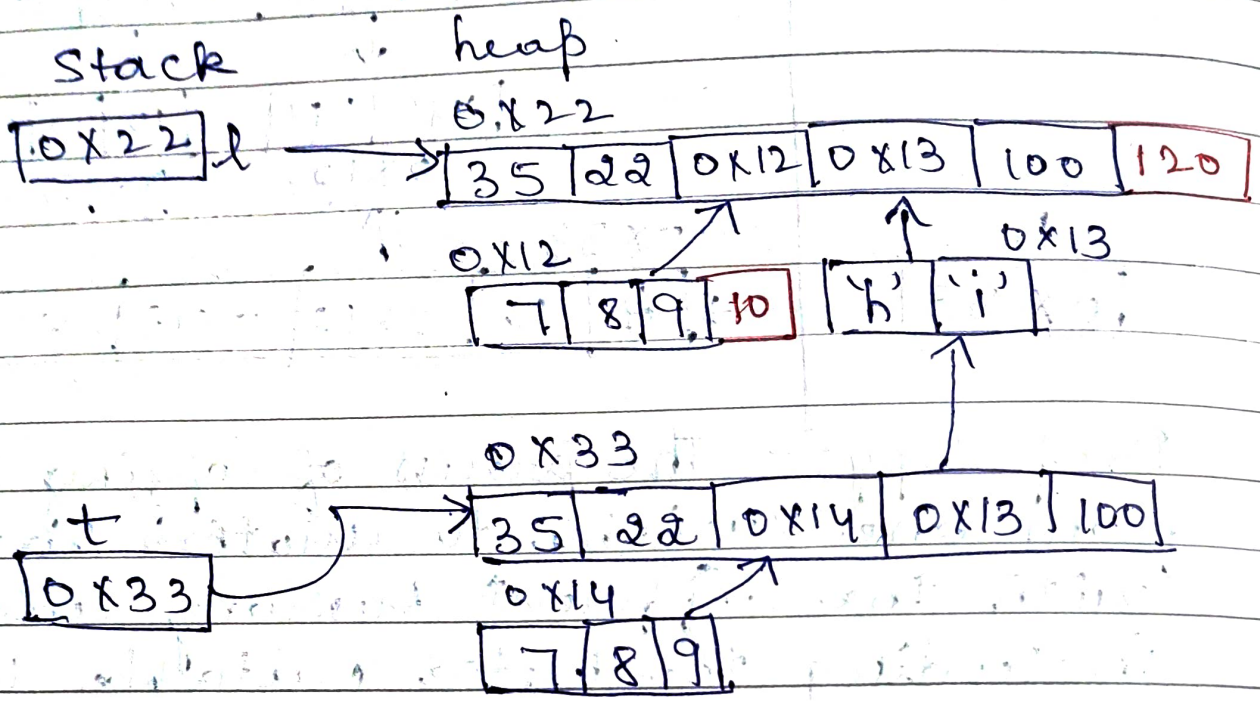
```
import Copy
dest = Copy.deepCopy(source_var).
```

Ex:-

```
l = [35, 22, [7,8,9], 'hi', 100]
t = Copy.deepcopy(l)
print(l)
→ [35, 22, [7,8,9], 'hi', 100]
print(t)
→ [35, 22, [7,8,9], 'hi', 100]
l.append(120)
print(l)
→ [35, 22, [7,8,9], 'hi', 100, 120]
print(t)
→ [35, 22, [7,8,9], 'hi', 100]
l[2].append(10)
print(l)
→ [35, 22, [7,8,9,10], 'hi', 100, 120]
```

print(t)

→ [35, 22, [7, 8, 9], 'hi', 100]



31/3/26

Data Types :- Data types are used to specify size & type of the value.

* Based on number of values are store in a variable.
* Datatypes are classified into two types :-

1. Single value / individual value
2. Multi value / Collection value.

1. In single value we have :-
   * Integer (int)
   * float
   * Complex
   * bool

   } numerical datatype.

2. In Multi value we have :-
   * String (Str)
   * list
   * tuple
   * Set
   * dictionary (dict).

1. Integer (int) :- It is a real number without decimal point. it can be both +ve & -ve.
* Every datatype will be having two types of values like default & non-default values.

* Default Value :- It is the inital value of a datatype which is internally Equally to false & Every data type Contains only one default value.
* To access a default value we have to call the datatype name.

Ex:- int() dtype()
→ 0 ↳ syntax

* Non-Default Value :- Apart from default rest all the values will be Consider as non-default value which is internally equals to True.
* The default value of integer is 0.

* type():- It is an inbuilt function used to access datatype of a value
Syntax :-
type (var/val).

Ex:- a = 2026
type (a)
→ <class 'int'>
type (2026)
→ <class 'int'>

* bool():- Used to check whether the value is default or non default. If the value is default the result will be False otherwise True.

2. Float :- float is a real number with decimal point. The default value of float is 0.0. After decimal point Maximum it takes upto 16 digits only

Ex:- x = 10.2
print(x)
→ 10.2
type(x)
→ <class 'float'>
bool(x)
→ True
float()
→ 0.0
bool(0.0)
→ False

.34
→ 0.34
345.
→ 345.0
.
→ Error

3. Complex :- A Complex is a Combination of real & imaginary numbers.

Integer/float
a ± bj (j = $\sqrt{-1}$)
real part
imaginary part

* The default value of Complex is 0j.
* Only J or j should be used as imaginary value.
* We should not change the position of b & j it will be considered as the variable & raises error.

2. Float :- float is a real number with decimal point. The default value of float is 0.0. After decimal point Maximum it takes upto 16 digits only

Ex:- x = 10.2
print(x)
→ 10.2
type(x)
→ <class 'float'>
bool(x)
→ True
float()
→ 0.0
bool(0.0)
→ False

.34
→ 0.34
345.
→ 345.0
.
→ Error

3. Complex :- A Complex is a Combination of real & imaginary numbers.

Integer / float
a ± bj (j = $\sqrt{-1}$)
real part
imaginary part

* The default value of Complex is 0j.
* Only J or j should be used as imaginary value.
* We should not change the position of b & j it will be considered as the variable & raises error.

* Writing an independent j is not allowed if it is mandatory to mention value for b.

Ex:- COM = 6+8j
COM
→ (6+8j)
type(COM)
→ <class 'complex'>
6+8J
→ (6+8j)
8J+6
→ (6+8j)
3+5k
→ Error : no other variable should use except J or j

4. bool :- It is datatype with only two values i.e, True & False
* True is considered as non-default value
* False is considered as default value
* Boolean values will be used as result of checking conditions.

Ex:- 10 > 5
→ True
10 < 5
→ False

1/4/26

Multi / Collection datatype :-

1. String :- String is a collection of Character enclosed blw single Quote (' ') or double Quote (" ") or triple Quote (''' ''').

* We can store upercase, lowercase, digits & also special character inside the string.
* Syntax :-

Var = 'Char1Char2 . . . . Charn' (or)
Var = " Ch1 Ch2 . . . Chn " (or)
Var = ''' Ch1 Ch2 . . . . Chn '''

* The default value of string is (' ').

Ex :- Str()
→ ' '

* Double Quote should be used when the string already contain single Quote in it.
* Triple Quote should be used to create Multiline string & also for Commenting purpose. It is also called as DOC string.

* len() :- It is a inbuilt function which is used to Count the number of values present in the Collection. It supports only Collection datatypes

Ex :- Str = " hello World ! "
len (str)
→ 12

Indexing :- It is a process of fetch each & every value from a Collection using its position or the sub address.

* There are 2 types of Indexing :-

1. Positive indexing :- The Counting starts from left side of the Collection. Always the first element starts with 0 index & the last element end with len(Collection) - 1 index.
2. Negative indexing :- The Counting starts from right side of the string. Always the last element starts with -1 index.

* It supports only Collection ~~with~~ (which are ordered or sequence.

* Syntax :-

var [index].

Ex :- S = 'hello world'

len(S)

→ 11

S[0]

→ 'h'

S[3]

→ 'l'

S[-3]

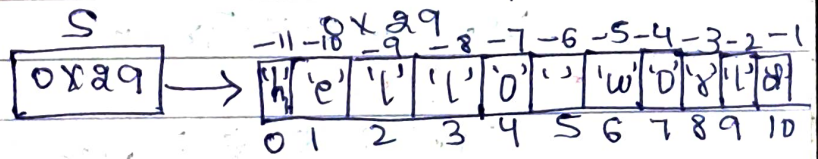
→ 'r'

S[-1]

→ 'd'

S[-6]

→ ' '



* Modify the String :- String is a immutable datatype. So it cannot be Modify.

* Mutable Values :- Which allows Modification like adding, deleting or updating the value.
Ex :- List, Set, dict.

* Immutable Values :- Which doesnot allow modification.
Ex :- String, triple, single value data type

5/4/26

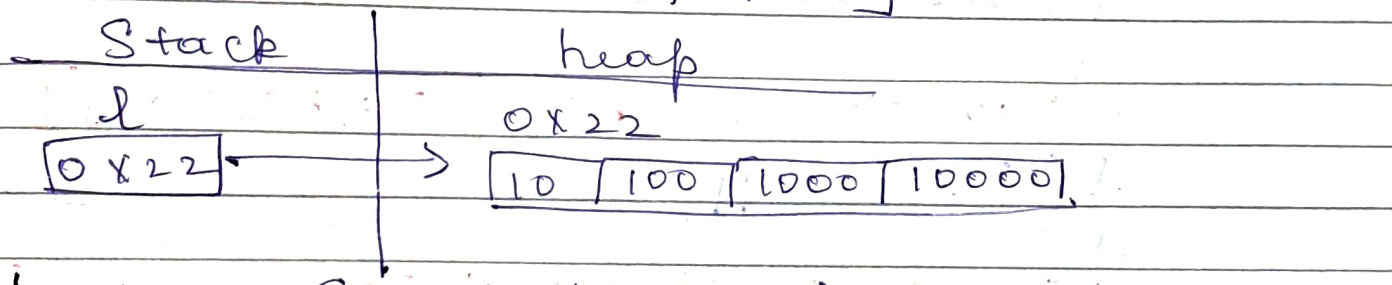
List Data type :- list is a collection of homogenous & heterogenous type of values.

* It is ordered collection data type.
* List can store Multiple data type in ordered Collection.
* List is Mutable Collections.
* Syntax :- [V1, V2, V3 ..... Vn]

Ex:- [10, 20, 30, 40] homogeneous.
[10, 20, 10.2, 'hii', 100] heterogeneous.

* Default value of list is [].

Ex:- l = [10, 100, 1000, 10,000]



* len() :- Count the number of values present in the list.

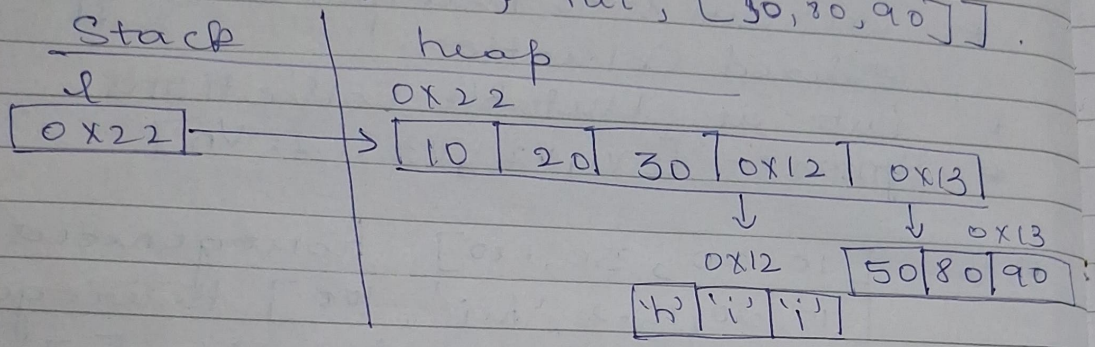
Ex:- l = [10, 20, 30, 'hii', 'bye', 100]
len(l)
→ 6

* Indexing :- Syntax : var[index]
* We can also modify the element based on the index value.

Ex:- l = [10, 20, 30, 'hii', 'bye', 100]
l[2]
→ 30
l[3] = 'happy'
→ l
[10, 20, 30, 'happy', 'bye', 100]

l[-4]
→ 30

Ex:- l = [10, 20, 30, 'hii', [50,80,90]].



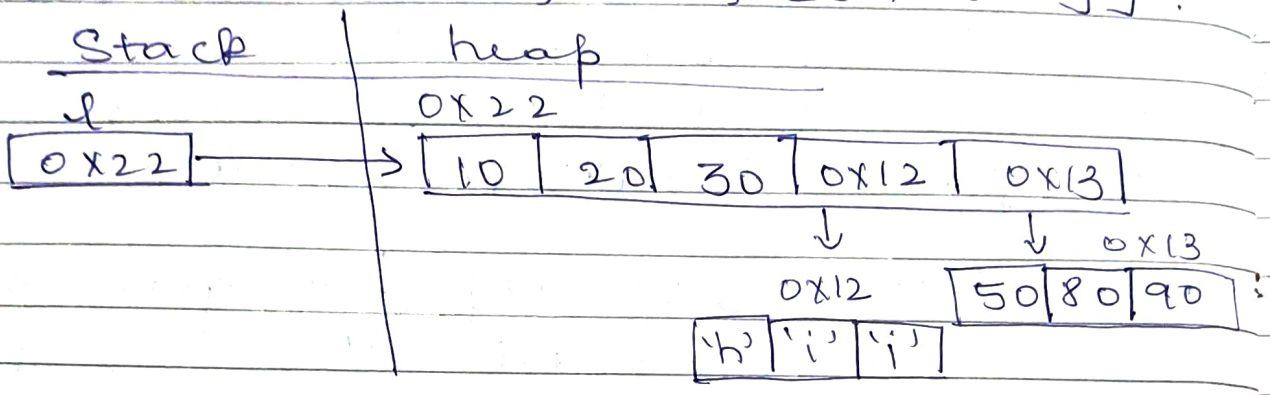
Ex:- l = [10, 20, 30, 'hii', [50,80,90]]
l[4][2] = 500
l
→ [10, 20, 30, 'hii', [50,80,500]]
l[3] = 'happy'
l
→ [10,20,30,'happy',[50,80,500]]
l[-1][1] = 800
l
→ [10,20,30,'happy',[50,800,500]].

List functions :-

1. append() :- It is used to append the element at the end of the list.
Syntax :-
var.append(value)
* It takes only one value at a time.

l[-4]
→ 30

Ex:- l = [10, 20, 30, 'hii', [50,80,90]]



Ex:- l = [10, 20, 30, 'hii', [50,80,90]]
l[4][2] = 500
l
→ [10, 20, 30, 'hii', [50,80,500]]
l[3] = 'happy'
l
→ [10,20,30,'happy',[50,80,500]]
l[-1][1] = 800
l
→ [10,20,30,'happy',[50,800,500]]

List functions :-

1. append() :- It is used to append the element at the end of the list.
Syntax :-
var.append(value)

* It takes only one value at a time.

Ex:- l = [12, 23, 34, 'open', 'close', [12, 13, 'good']]
l.append(1000)
→ [12, 23, 34, 'open', 'close', [12, 13, 'good'], 1000]]

2. Insert():- Based on Mentioned index it adds the value.
Syntax:-
var.insert(index, value).
Ex:- l = [12, 23, 34, 'close', [12, 13, 'good'], 1000]]
l.insert(5, 'bad')
l
→ [12, 23, 34, 'close', [12, 13, 'good'], 'bad', 1000]]

3. Extend():- To add multiple value to the list at the end.
Syntax:-
var.insert([val1, val2 ..... valn]).
Ex:- l = [12, 23, 34, 'open', 'close', 'bad', [12, 13, 'good'], 1000]
l.extend([[0, 22]])
l
→ [12, 23, 34, 'open', 'close', 'bad', [12, 13, 'good'], 1000, 0, 22]
l.extend('go')
l
→ [12, 23, 34, 'open', 'close', 'bad', [12, 13, 'good'], 1000, 0, 22, 'g', 'o']

```
l.extend([['come']])
l
→ [12, 23, 34, 'open', 'close', 'bad',
[12, 13, 'good'], 1000, 0, 22, 'g', 'o',
'come']
```

4. POP() :- It is used to remove the last element from the list.

Syntax :-

```
var.~~name~~ pop()
```

* To remove particular element by using index.

Syntax :-

```
var.pop(index)
```

Ex :-
```
l = [50, 60, 'go', [12, 13, 14]]
l.pop()
→ [12, 13, 14]
l
→ [50, 60, 'go']
l.pop(1)
→ 60
l
→ [50, 'go']
```

5. remove() :- It is used to remove particular value in a list.

Syntax :-

```
var.remove(val)
```

Ex:- S=['go', 'come', 11, 12]
S.remove('go')
S
→ ['come', 11, 12]

6. clear() :- It clears the all the elements from the list.
Syntax :- list.clear()

Ex:- S=['go', 'come', 11, 12]
S.clear()
S
→ []
del S
S
→ Empty. doesn't print anything.

7/4/26

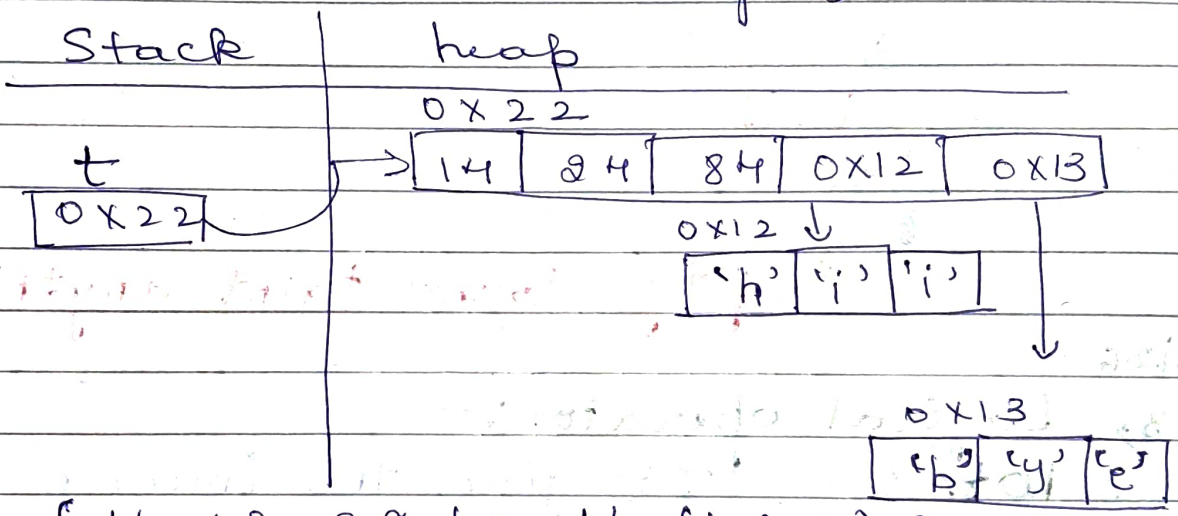
Tuple Datatype :- Tuple is a ordered collection. It is also a collection of homogeneous & heterogeneous type of values.

* Tuple is immutable datatype.
* Syntax :-

var = (val1, val2 .... valn) (or)
val1, val2 .... valn

* Default value of tuple is ( ).

Ex :- t = (14, 24, 84, 'hii', 'bye')



Ex :- t = (14, 12, 99, 'good', (1, 2, 3), 'ok')

t[4]

→ (1, 2, 3)

t1 = (40)

type(t1)

→ <class 'int'>

t1 = (40,)

type(t1)

→ <class 'tuple'>

t2 = 1, 2, 3, 'yes', 100

t2

→ (1, 2, 3, 'yes', 100).

tuple()

→ ()

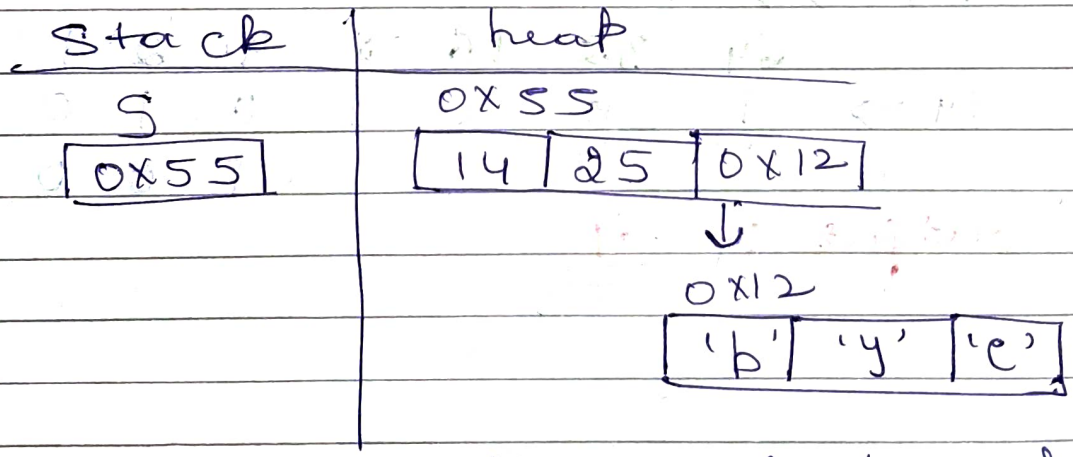
8/4/26

Set datatype :- It is a Heterogenous & heterogenous Collection of values.

* It is a unordered collection & it doesnot allow duplicate values.
* It is always enclosed with { }
* Indexing is not possible and also fetching the values & modification is also not possible in set due to unordered Collection.
* Set is a mutable datatype.
* Syntax :-

  var = {val1, val2 ..... valn}
* default value is 'set()'

Ex:- S = {14, 25, 'bye'}



* It will not allow mutable values inside set such as (list, set, dict).

Ex:- S = {4, 5, 6 ['hello', 45], 100}

→ error.

{4, 5, 6, [ ], { }, { }}

Functions to add values in Set :-

1. add() → It adds the value to random position.

Syntax :- var.add(value)

Ex:- S1 = {1, 2, 3, 'good'}
S1.add(100)
S1
→ {1, 2, 3, 100, 'good'}

2. update() → Multiple values randomly.

Syntax :- var.update(Collection)

Ex:- S1.update([10, 20])
S1
→ {1, 2, 3, 100, 10, 'good', 20}

Functions to delete the values :-

1. Pop() :- removes the first value from the ~~list~~ set.

Syntax :- var.pop()

Ex:- S1.pop
→ 1
S1
→ {2, 3, 100, 10, 'good', 20}

2. remove() :- removes specified values.

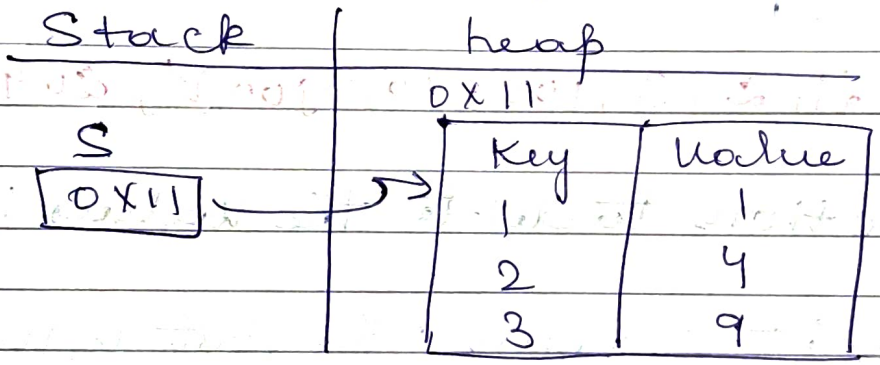
Syntax :- var.remove(value)

Ex:- S1.remove(100)
S1
→ {2, 3, 10, 'good', 20}

3. clear():- removes all the values.
Syntax:- var.clear()
Ex:- S1.clear()
S1
→ Set ().

Dictionary datatype:- Collection of key value pair enclosed blw {}.
* Syntax:-
var = {key1 : val1, key2 : val2 ...}
* Default value {}.

Ex:- S = {1:1, 2:4, 3:9}



* To access the value we use key.
Syntax:- var [key].
Ex:- S = {1:1, 2:4, 3:9}
S [1]
→ 1

* Dictionary values can be modify but key ~~values~~ cannot be modify.
and key should not be duplicate.

Ex:- S = {1:1, 2:4, 3:9}
S[1] = 5
S
→ {1:5, 2:4, 3:9}

S1 = {1:5, 2:4, 3:9, 1:7}
S1
→ {1:7, 2:4, 3:9}

* Values can be anything but keys must be immutable & unique.

Functions:-

1. add():- It is used to key & value.
Syntax:- var[key] = value.

Ex:- S1 = {1:5, 2:4, 3:9}
S1[4] = 16
S1
→ {1:5, 2:4, 3:9, 4:16}

2. delete the values:-
* Pop():- It is used to pop the element based on key values.
Syntax:- var.pop(key).

Ex:- S1.pop(4)
→ 16

3. Popitem():- It is used to delete last key value pairs

1/4/26

Operators :- This are the special symbols used to perform a specific task or operations.

Operands :- operands are the value required to perform the operations.

There are 7 types of operators :-

1. Arithemetic operator
2. Relational | Comparision operator.
3. Logical operator
4. Assignment operator
5. Bitwise operator
6. Membership operator
7. Identity operator

1) Arithemetic operators :- These are used to perform basic mathematical operation There are types :-

1. Addition (+)
2. Substraction (-)
3. Multiplication (*)
4. Division
   - True division (/)
   - floor division (//)
   - Modulus (%)
5. Power (**).

1. Addition operator :- whenever we use + on single value datatypes it gives the sum of the operands & when

We use on Collection data type it works as a Concatenation operator.

* Syntax:-
  op1 + op2
* on single values irrespective of the data type it gives the sum of both the operands. But in case of collections both the operands must be of same datatypes.
* It doesnot support set & dict.

Ex:- 10+20
→ 30

10.5+35
→ 45.5

4+5j+4
→ (8+5j)

4+5j+4+6j
→ (8+11j)

True+10
→ 11

False+10
→ 9

'hii'+'hello'
→ 'hiihello'

[10,20,30]+[20,30,40,50]
→ [10,20,30,20,30,40,50]

(10,20,40,60)+(10,12,13)
→ (10,20,40,60,10,12,13)

[10,20,30]+(1,2,3)
→ Error

'hi'+[40,50,60]
→ error

2. Subtraction :- It supports all the single value datatypes & gives the difference b/w the operands irrespective of the datatype.

* In Collections datatype, it supports only sets. Here, it removes the common elements in both the operands & the result will be remaining elements from set 1.

* Syntax :-

op1 — op2

Ex:- 20 − 10
→ 10

20.5 − 10
→ 10.5

4 − 2 + 3j
→ (2 + 3j)

5 − True
→ 4

'hello' − 'hi'
→ error

[4, 5, 6, 7] − [8, 9, 10]
→ error

(2, 3, 4) − [4, 6, 7, 9]
→ error

{3, 2, 4, 5} − {4, 5, 6, 7}
→ {3, 2}

{4: 2} − {5: 3}
→ error

{4, 5, 6, 7} − {6, 7, 8}
{4}

3. Multiplication (*):- It supports all the single value datatypes & it gives product of the operands.
* It doesnot support set & dict.
* while multiplying collection data type any one operand must be an integer

Ex:- 10 * 20
→ 200

10 * 5.5
→ 55.0

2 * 3+2j
→ (6+2j)

2 * (4+3j)
→ (8+6j)

'hii' * 'bye'
→ error

'hi' * 2.5
→ error

2 * 'bye'
→ 'byebye'

[1,2,3] * 2
→ [1,2,3,1,2,3]

(4,5,6) * 2
→ (4,5,6,4,5,6)

{4,2,3} * 2
→ error

{4:2,5:2} * 2
→ error

True * False
→ 0

4. Division :- Division operator supports only single value datatype.
* There are 3 types:-
1. True division (/)
2. Floor division (//)
3. Modulus (%).

1. True division (/):- It supports all the single value data types & returns the result as the exact Quotient

2. Floor division (//):- It supports all the single value except Complex & the result will be the quotient without decimal part.

3. Modulus operator (%):- It supports all the single value except Complex & the result will be the remainder.

Ex:- 10/3
→ 3.33333

10//3
→ 3

10%3
→ 1

25.5/2
→ 2.125

23.4//2.5
→ 9.0

23.4%2.5
→ 0.8999999986

(4+5j)/(2+5j)
→ (1.137931034 − 0.3448275862)

(4+5j)//(2+5j)
→ error

(4+5j)%(2+5j)
→ error

[4,2,3]/2
→ error

'hi'/3
→ error

2/4/26

5. Power (**) :- It is used to find the exponential values. It supports only single value data types.

Ex:- 2**5

→ 32

3.5**3

→ 42.875

(2+3j)**2.5

→ (-19.122475853+15.609903 55j)

4.56**True

→ 4.56

4.56**False

→ 1.0

True**False

→ 1

'hi'**2

→ error

2) Relational operator :- These operator are used to Compare the operands & build the Conditions. Always the result will be boolean values. If the Condition is satisfied the result will be True else the result will be False.

* There are 6 types :-

1. Relational equal to (==)
2. Not equal to (!=)
3. Greater than (>)
4. Lesser than (<)
5. Greater than or equal to (>=)
6. Lesser than or equal to (<=).

1. Relational equal to (==) :- It gives the result has True if both the operands are exactly equal. else result will be False.

Ex:- 10 == 10.5
→ False
10 == 10.0
→ True
'hi' == 'hii'
→ False
'hi' == ['hi']
→ False
'hi' == ['hi'][0]
→ True

{4,5,6} == {5,6,4,5}
→ True
[4,5,6] == [6,4,5]
→ False
(4,5,6) == (5,6,4)
→ False
[4,5,6] == [4,5,6]
→ True
{4:2, 5:2} == {4:5, 5:3}
→ True

2. Not equal to (!=) :- It gives result as True if the operands are not equal or else False.

Ex:- 'hello' != 45
→ True

'hello' != 'hello'
→ False

[1, 2 3, 4] != (1, 2, 3, 4)
→ True

[1,2,3,4] [1] != (1, 2, 3, 4) [1]
→ 2 != 2
→ False

6/4/26

3. ~~Equivalent to~~ Greater than (>) :- It gives result as true if operand 1 is greater than operand 2. It does not support Complex & dict.

* For single values irrespective of the datatypes it Compares except Complex. But for Collection datatype both the operands must be of same data type.

* Strings will be Compare based on the ASCII value. If the ASCII value of first Character of operand 1 is greater than first Character of Operand 2 the result will be True. If the ASCII value of op1 of 1st Character is less than op2 of 1st

character result will be false. If the characters are same it compares the 2nd character of both the operands of it continues.

* on list & tuple value by value comparision will be done.
* On set, if op1 is superset of op2 the result will be true.

* Ord() :- It is use to accus the ASCII value of a character.
* Syntax :- Ord('c').

Ex:-
10.0 > 10
→ False
1 > False
→ True
10 > 20
→ False
3+4j > 2+5j
→ error
'hello' > 123
→ error
'hello' > 'hello'
→ False
'hii' > 'bye'
→ True
'Hello' > 'hi'
→ False
ord('a')
→ 97

[40,20,30] > 'hii'
→ error
[40,20,30] > [45,25,13]
→ False
[40,55,60] > [40,60,70]
→ False
(40,20,10) > (50,40,20)
→ False
{10,14,28} > {14,28}
→ True
{10,13,12,2} > {20,30}
→ False
{10:20} > {40:30}
→ error
'hello' > 'hell'
→ True
[40,20,'hi'] > [20,'hi',30]
→ True

4. Greater than or equal to :- It gives result has true if operand 1 is greater than op2 or if both the operands are equal.

Ex:- 10 >= 9
→ True
10.5 >= 10.5
→ True
[4,5,6][1] >= [1,2,3][1]
→ True
{4,5,6,7,8} >= {6,7,8}
→ True
{10,20,30} >= {20,10,30}
→ True
[6,7,8,9] >= [4,3,2,1]
→ False

5. Lesser than (<) :- It gives result has true if op1 is lesser than op2.
* It doesnot support Complex & dict datatype.
* In set, the 1st set should be a subset & 2nd set should be a superset

Ex:- 10 < 20
→ True
10.4 < 11
→ False
4+3j < 5+6j
→ error

[4,5,6] < [6,7,8,9]
→ True
{1,2,3} < {4,1,3,2}
→ True

6. Less than or equal to (<=):- It gives result as true if op1 is lesser than op2 or if both the operands are equal.

Ex:- 10 <= 20
→ True
[4, 6, 7, 8] <= [4, 6, 9, 8]
→ True
{6, 7, 8} <= {8, 9, 10, 6, 7}
→ True

3) Logical Operator:- This operators are used to combine Multiple Conditions.

* There are 3 types:-
1. and 2. or 3. not

1. and :- when the operands are Conditions it gives results as true if all the Condition are satisfied otherwise false.

Truth table:-

Cond1	Cond2	Result
T	T	T
T	F	F
F	T	F
F	F	F

* If the operands are values it works differently :-
1. if op1 is non default → op2
2. if op1 is default → op1.

Ex:- 18 < 70 and 'hello' > 'bye'
→ True
1 == 1.0 and 0 >= 0.0 and 98 > 76
→ True
10 and 344
→ 344
0 and 23
→ 0
1 > 10 and 3+2j < 4+6j
→ False
10 < 20 and 3+4j > 6+7j
→ error

2. Or :- when the operands are conditions it gives result as true if any one condition is satisfied otherwise false.

Truth table :-

Cond1	Cond2	Result
T	T	T
T	F	T
F	T	T
F	F	F

* If the operands are values it works differently such as :-
1. if op1 default → op2
2. if op1 non-default → op1

Ex:- 12 < 34 or 2+3j > 6
→ True

12 > 34 or 6 < 50
→ False

12 or 45
→ 12

0 or 2.4 or 0 or [] and 78
→ []

0 or 2.4 or 0 or [] or ()
→ ()

7/4/26

3) Logical operator :-

3. not :- It gives the inverse of the result.

Ex :- not (10 > 1)

→ False

not (1 > 23)

→ True

not (99) if it is non default → False

→ False

not (0) if it is default → True.

→ True

8/4/26.

4) Assignment operator (=) :- This are used to assign a value to a variable.

* Compound [augmented assignment operator :- This are used to update the existing variables.

a = a+b	a+=b
a = a-b	a-=b
a = a*b	a*=b
a = a/b	a/=b
a = a||b	a||=b
a = a%b	a%=b
a = a**b	a**=b.

Ex:- a = 10
b = 20
a+=b
a
→ 30
a-=b
a
→ 10

a = 50
b = 10
a/=b
a
→ 5.0

* a = 2
b = 3
a**=b
a
→ 8

5) Bitwise operator :- It converts the number into binary & performs the required operations on each & every bit.

* There are 6 types :-

1. Bitwise and (&)
2. Bitwise or (|)
3. Bitwise not (~)
4. Bitwise XOR (^)
5. Bitwise left shift (<<)
6. Bitwise right shift (>>)

8/4/26

4) Assignment operator (=) :- This are used to assign a value to a variable.

* Compound (augmented assignment operator :- This are used to update the existing variables.

a = a+b	a+=b
a = a-b	a-=b
a = a*b	a* =b
a = a/b	a/ =b
a = a||b	a|| =b
a = a%b	a% =b
a = a**b	a** = b.

Ex:-

```
a = 10
b = 20
a+=b
a
```
→ 30

```
a-=b
a
```
→ 10

```
a = 50
b = 10
a/=b
a
```
→ 5.0

```
a* a = 2
   b = 3
   a**=b
   a
```
→ 8

5) Bitwise operator :- It Converts the number into binary & performs the required operations on each & every bit.

* There are 6 types :-

1. Bitwise and (&)
2. Bitwise or (|)
3. Bitwise not (~)
4. Bitwise XOR (^)
5. Bitwise left shift (<<)
6. Bitwise right shift (>>)

1. Bitwise and (&) :- It supports only integer & set.
   * On Integer, it converts them into binary & performs bit by bit 'and' operation.
   * on Set, it gives the common elements from both the set.

Ex:- 1) 25 & 8

	16	8	4	2	1
25 →	1	1	0	0	1
8 →	0	1	0	0	0
	0	1	0	0	0

(&)

→ output :- 8.

2) 60 & 75

	64	32	16	8	4	2	1
60 →	0	1	1	1	1	0	0
75 →	1	0	0	1	0	1	1
	0	0	0	1	0	0	0

(&)

→ output :- 8

3) {4, 3, 2, 6, 7} & {8, 9, 3, 2, 6}
→ {3, 2, 6} (Common elements).

2. Bitwise or (|) :- It supports only Integer and set.
* On Integer, it Converts them into binary & performs bit by bit 'or' operation.
* On Set, it gives unique elements from both the set.

Ex:- 1) 42 | 38

	32	16	8	4	2	1	
42 →	1	0	1	0	1	0	(or)
38 →	1	0	0	1	1	0	
	1	0	1	1	1	0	

→ output :- 32 + 8 + 4 + 2 = 46.

2) 50 | 82

	64	32	16	8	4	2	1	
50 →	0	1	1	0	0	1	0	(or)
82 →	1	0	1	0	0	1	0	
	1	1	1	0	0	1	0	

→ output :- 64 + 32 + 16 + 2 = 114.

3) {1, 2, 6, 7, 8} | {10, 9, 3, 4}
→ {1, 2, 3, 4, 6, 7, 8, 9, 10}.

3. Bitwise XOR (^) :- On Integer, it Converts the integer into binary & perform bit by bit 'XOR' operation.
* On Set, it gives the elements from both the sets by removing the common elements.

Ex:-1) 96 ^ 22

	64	32	16	8	4	2	1	
96 →	1	1	0	0	0	0	0	(XOR)
22 →	0	0	1	0	1	1	0	
	1	1	1	0	1	1	0	

→ output :- 64 + 32 + 16 + 4 + 2 = 118

2) 88 ^ 66

	64	32	16	8	4	2	1	
88 →	1	0	1	1	0	0	0	(XOR)
66 →	1	0	0	0	0	1	0	
	0	0	1	1	0	1	0	

→ output :- 16 + 8 + 2 = 26

3) {1, 2, 3, 4, 5} ^ {3, 4, 5, 6, 7}
→ {1, 2, 6, 7}

4. Bitwise not (~) :- It supports only integer datatype and it take only one operand. It is used to inverse the operand.

* Syntax :- −(op+1).

Ex:- ~26 → −(26 + 1)
−27 = −27 ✓

~99 → −(99 + 1)
−100 = −100

5. Bitwise leftShift (<<) :- It only support Integers. It converts integer to binary & shift the bit towards left side by specified number of times.

Ex:- 12 << 3

64 32 16 8 4 2 1

1 1 0 0

1 1 0 0 0 0 0

1 1 0 0 0 0 0

→ output :- 64+32 = 96

(or)

12 → 1 1 0 0 (Add 3 zeros)

1 1 0 0 0 0 0

8 4 2 1

12 → 1 1 0 0 (Add 3 zeros)

1 1 0 0 0 0 0

64 32 16 8 4 2 1

→ output :- 16 + 32 = 96

6. Bitwise RightShift (>>) :- It only support Integers. It converts integer to binary & shift the bit towards right side by specified number of times.

Ex:- 90 >> 5

64 32 16 8 4 2 1

90 → 1 1 0 1 1 0

1 1 0

→ output :- 2+1 = 3.

Assignment :-

1) 135 & 67
2) 88 & 39
3) 125 | 65
4) 120 ^ 140
5) 65 << 5
6) ~4 >> 4

1. 135 & 67

	128	64	32	16	8	4	2	1	
135 →	1	0	0	0	0	1	1	1	} (&)
67 →	0	1	0	0	0	0	1	1	
	0	0	0	0	0	0	1	1	

→ output :- 2+1 = 3

2. 88 & 39

	64	32	16	8	4	2	1	
88 →	1	0	1	1	0	0	0	} (&)
39 →	0	1	0	0	1	1	1	
	0	0	0	0	0	0	0	

→ Output :- 0

3. 125 | 65

	64	32	16	8	4	2	1	
125 →	1	1	1	1	1	0	1	} (|)
65 →	1	0	0	0	0	0	1	
	1	1	1	1	1	0	1	

→ output :- 64+32+16+8+4+0+1 = 125

4. 120 ^ 140

	128	64	32	16	8	4	2	1	
120 →	0	1	1	1	1	0	0	0	(A)
140 →	1	0	0	0	1	1	0	0	
	1	1	1	1	0	1	0	0	

→ Output :- 128 + 64 + 32 + 16 + 4 = 244.

5. 65 << 5

	64	32	16	8	4	2	1					
65 →	1	0	0	0	0	0	1					
	1	0	0	0	0	0	1	0	0	0	0	0
	2048	1024	512	256	128	64	32	16	8	4	2	1

→ Output :- 2048 + 32 = 2080

6. 74 >> 4

	64	32	16	8	4	2	1
74 →	1	0	0	1	0	1	0
				→	1	0	0

output :- 4

9/4/26.

6) Membership operator :- It is used to check whether the value is present in the collection or not.

* There are two types :-
1. in
2. not in.

[4, 5, 6]
6 → true

1. in operator :- It gives result as true, if value is present in the collection.

Syntax :-
value in Collection.

* when we use in operator on string, both the operands must be of string datatype itself.
* For set & dict, first operand must be an immutable datatype.

Ex :-
1 in 123
→ error

1 in [4, 5, 1, 7]
→ True

'e' in 'hello'
→ True

e in 'hello'
→ ~~False~~ error

'7' in 'bycc23'
→ False

[11, 22] in {1:2, 3:4}
→ error

11 in {1:2, 3:4}
→ False

[1, 2, 3] in {1, 2, 3, 4}
→ error

2. not in operator :- It gives result as true, if value is not present in the collection.

Syntax :-

value not in Collection.

Ex :- 1 not in [4, 5, 6]

→ True

'hello' not in 'hello welcome'

→ False

7) Identity operator :- It is used to check whether the values are pointing to same memory address or not.

* There are 2 types :-

1. is
2. is not

* when we store same values different variables all the mutable values takes different memory address & all the immutable values takes same memory address.

1. is operator :- It gives result as true if the values are pointing to same memory address.

Syntax :-

val1 is val2

2. is not operator :- It gives result as True if the values are not pointing to same memory address.

Syntax :-

val1 is not val2.

Ex :-

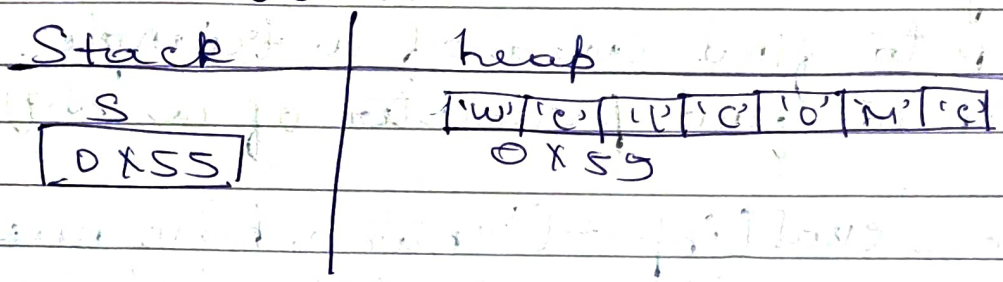
```
a = 50
b = 50
a is b
→ True

a = 40.8
b = 40.8
a is b
→ True

a = 50
b = 48
a is not b
→ True

a = 'hello@#'
b = 'hello@#'
a is b
→ True

l1 = [1, 2, 3, 4]
l2 = [1, 2, 3, 4]
l1 is l2
→ False
```

10/4/26.

Slicing :- It is a process of accessing/fetching multiple/part of a collection values.

* Syntax :- var [Staring index : end index : Step]
* It Supports only str, tuple, list.

Ex :- S = 'Welcome'



Ex :- S = 'Welcome to Python'
(0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16)

S [0 : 7 : 1]
→ 'Welcome'

S [6 : -1 : -1]
→ ' '

S [11 : 17 : 1]
→ 'Python'

S [16 : -1 : -1]
→ ' '

S [-1 : -18 : -1]
→ 'emoclew'

Simplified form of Slicing :-

1. Starting Index = 0 or -1
   var [: EI : Step]

Ex:- S[:12:1]
→ 'WelCome to P'

2. Ending Index = 0 or -1
var [SI::Step]
Ex:- S[4::2]
→ 'oet yhn'

3. Step Size = 1
var [SI:EI]
Ex:- S[7:12]
→ ' to p'

Type Casting:- Converting from one datatype to another type is called Type Casting.

1. int

source_type	destination_type
int	float
	Complex
	bool
	str
	list
	tuple
	set
	dict

list, tuple, set, dict — not possible

Ex:- S = 10
float(S)
→ 10.0
Complex(S)
→ (10+0j)
bool(S)
→ True
Str(S)
→ '10'

2. Float

Source_type	destination_type
float	int
	Complex
	bool
	Str
	list
	tuple
	Set
	dict

(list, tuple, Set, dict) → X

Ex:- S = 10.7
int(S)
→ 10
Complex(S)
→ (10.7+0j)
bool(S)
→ True
Str(S)
→ '10.7'

3. Complex

Source_type	destination_type
Complex	bool
	Str

Ex:- C = 10+6j

C
→ (10+6j)
bool(C)
→ True
Str(C)
→ '(10+6j)'

4. bool

Source_type	destination_type.
bool	int
	float
	Complex
	bool
	str

Ex:- b = True

int(b) → 1
float(b) → 1.0
Complex(b) → (1+0j)
Str(b) → 'True'

5. String :-

Source type	destination_type.
Str	1) int (Str can be convert into int only the digit without decimal not a Character).

Ex:- S = 'Hello'
int(S)
→ error
S1 = '10'
int(S)
→ 10
S1 = '10.7'
int(S1)
→ error

2) float (Str can convert into float only digits not a char with & without decimal)

Ex:- S = "10"
float(S)
→ 10.0
~~float~~ S = "10.9"
float(S)
→ 10.9

3) Complex (Str can be convert into Complex only the digit with or without decimal)

Ex:- U = '2+9j'
u
→ (2+9j)
Complex(U)
→ (2+9j)
S = '34.9'
→ (34.9+0j)

S = '67'
Complex(S)
→ (67+0j)
S = 'j'
Complex(S)
→ 1j

4) boolean (Str can Convert into bool)

Ex:- Str = 'Hello'
bool(Str)
→ True

Str = ' '
bool(Str)
→ False

5) list (~~list~~ can be Convert into list)

Ex:- Str = 'hello'
list(Str)
→ ['h', 'e', 'l', 'l', 'o']
Str = "35.5"
list(Str)
→ ['3', '5', '.', '5']

6) Tuple(Str can Convert into tuple)

Ex:- Str = 'bye'
tuple(Str)
→ ('b', 'y', 'e')

7) Set(Str can Convert into Set)

Ex:- Str = 'OK'
Set(Str)
→ {'K', 'O'}

8) dict(Str doesnot support dict)

6. List :-

1. int
2. float
3. Complex

} (list can be Convert) not

4. bool (list can convert into bool)

Ex:- list = ['4', '5', 'hi']
bool (list)
→ True
list = []
bool (list)
→ False

5. Str (list can convert into Str)

Ex:- list = ['10', '20', 'bye']
Str (list)
→ "['10', '20', 'bye']"
len (list)
→ 3
len (Str (list))
→ 19

6. tuple (list can convert into tuple).

Ex:- list=['4.2', '10', 10k]
tuple (list)
→ ('4.2', '10', '0k')

7. set (list can convert into set)
(which is unordered & removes duplicate

Ex:- list = ['10', '40', '0k']
set (list)
→ {'40', '0k', '10'}
list = ['10', '20', '10']
set (list)
→ {'20', '10'}

8. dict (list can be convert into dict only when list is having nested list otherwise error)

Ex:- list = ["42", "52", 'good']
dict (list)
→ error
list = [[1,2], (9,2), 6, 'Hi', {12,34}]
dict (list)
→ { 1:2, 9:2, 'H':'i', 12:34 }

7. Tuple:-
1. int (tuple cannot be convert into int)
2. float (" ")
3. Complex (" ")
4. bool (tuple can be convert into boo)

Ex:- t = ("4", "5", 'OK')
bool (t)
→ error

5. Str (Tuple can be convert into Str)

Ex:- t = (4, 5, 'OK')
Str (tuple)
→ "(4, 5, 'OK')"

6. list (Tuple can be convert into list)

Ex:- t = (6, 8, 9)
list (t)
→ [6, 8, 9]

7. Set (Tuple can be convert into Set)

Ex:- t = (4, 5, 9, 10, 5)
Set (t)
→ (9, 5, 4, 10)

8. dict (Tuple can be convert to dict only when tuple is having nested collection otherwise not).

Ex:- Tuple = ((42,52), 'by', [67,71])
dict (tuple)
→ {42:52, 'b':'y', 67:71}

8. Set :- Same as list, tuple. but when come to dictionary, set can convert into dict only when set is having nested collection otherwise not. And also it gives error when we store mutable values only because set is also mutable).

Ex:- Set = {(42, 25), 'Hi', 'OK'}
dict (set)
→ {42: 25, 'H':'i', 'O':'K'}
Set1 = {[99,42], 'Hi', 'bye'}
dict (Set1)
→ error.

9. Dictionary :- Same as list & tuple.

1. Str (Can Convert into dict)

Ex:- dict = {1:2, 4:6}
Str (dict)
→ '{1:2, 4:6}'

2. list (dict can Convert into list)

Ex:- dict = {42:3, 'h':'i', 66:2}
list(dict)
→ [42, 'h', 66]
* It takes only key.

3. tuple (dict can be Convert into tuple)

Ex:- dict = {43: 3, 'h': 'i', 66: 2}

tuple(dict)

→ (43, 'h', 66)

4. Set (Set can be Convert into dict)

Ex:- dict = {42: 3, 'h': 'i', 66: 2}

Set(dict)

→ {'h', 42, 66}

Input() :- It is used to take input from the user by default it takes every input in the form of string.

* Syntax :-

```
var = input('msg')
```

* Here, 'msg' is optional for better readability purpose we have to mention the message.
* To take the required input we have to type caste the input

function.

* Syntax:-
  var = datype(input('msg'))
* It works well for all the single value datatype but not for Collections. To take the input in the form of Collection we should use 'eval()' function.
* eval() :- To take input in the form of Collection.
  Syntax:-
  var = eval(input('Msg'))
* eval is capable of taking any type of input. It evaluates the data & stores it in its original type.

Ex:-* S = input("Enter a name:")
enter a name: Kavana
S
→ 'Kavana'

* S = input("enter a num:")
enter a num: 456
S
→ '456'

* i = int(input("enter a num:"))
enter a num: 123
i
123

* c = complex(input("enter a num:"))
enter a num: j
c
→ 1j

* l = list(input("enter list:"))
enter list: [4, 5, 6]
l
→ ['[', '4', ',', '5', ',', '6', ']']

* l = eval(input("enter list:"))
enter list: [4, 5, 6]
l
→ [4, 5, 6]

* d = eval(input("enter dict:"))
enter dict: {1:1, 2:4}
d
→ {1:1, 2:4}

* a = input()
123
a
→ '123'

10/4/26

Print():- It is an inbuilt function used to display the output.

Syntax:-

print(val, val, ... valn, end='\n', sep=' ')

Ex:-
print(10, 20, 30)
print('Hello everyone')
print(99, 999)

→ 10 20 30
Hello everyone
99 999

Ex:- print (10, 20, end = ' ')
print ('Hello', 'end = ' ')
print (99, 100)
→ 10 20 Hello 99 100

Ex:- print (10, 20, 30, sep = '*')
→ 10*20*30

Ex:- print (10, 20, end = ' ', sep = '*')
print ('OK')
→ 10*20 OK

Programs:-

1. WAP to display airthemetic operations:-

→
```
x = int(input('enter a num:'))
y = int(input('enter a num:'))
print('the airthemetic operations are:',
      x+y, x-y, x*y, x/y)
```

→ enter a num: 10
enter a num: 5
The airthemetic operations are:
15 5 50 2

2. WAP to reverse a String:-

→
```
s = input('enter a String:')
print('The reverse is:', s[::-1])
```

→ enter a string: bye
The reverse is: eyb

3. WAP to display area of rectangle.

→
```
l = eval(input('enter length :'))
b = eval(input('enter breadth :'))
print('The area of rectangle is :',
      l * b)
```

→
```
enter length : 10
enter breadth : 2
The area of rectangle is : 20
```

4. WAP to cal the Simple intrest.

→
```
P = int(input('enter principle amount :'))
t = int(input('enter duration :'))
r = int(input('enter rate :'))
print('Simple intrest is :', (P * t * r)/100)
```

→
```
enter principle amount : 40,000
enter duration : 10
enter rate : 3
Simple interest is : 12000.0
```

5. WAP to swap two num without Using third variable :-

→
```
a = eval(input('enter a num :'))
b = eval(input('enter b num :'))
print('The swap of two number is :',
      a, b = b, a)
```

→
```
enter a num : 10
enter b num : 20
The swap of two number is : a = 20,
b = 10
```

a = 10	a = a+b	b = a-b	a = a-b
b = 20	a = 10+20	b = 30-20	a = 30-10
a = a+b	a = 30	b = 10	a = 20
b = a-b			
a = a-b			

→ a = 20 , b = 10.

Assignment:-

1. WAP to find area of a triangle.
2. WAP to find perimeter of a square.

1.
```
base = eval(input('enter a base :'))
height = eval(input('enter a height :'))
print('The area of triangle is :',
      1/2*base*height)
```

→ enter a base : 12.34
enter a height : 45
The area of triangle is : 277.65.

2.
```
P = eval(input('enter a number :'))
print('The perimeter of square is :',
      4*P)
```

→ enter a number : 277.4
The perimeter of Square is : 1109.6

11/4/26

Control Statement :- This are used to Control the flow of execution of the program.

* There are 2 types :-
1. Conditional statement
2. Looping statement.

(1). Conditional Statement :- It is a type of Control statement which Controls the flow of execution of the program based on a Condition.

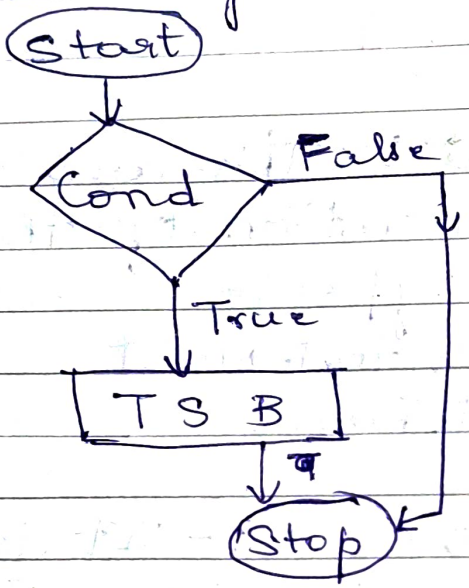
* There are 4 types :-
1. Simple if
2. if else
3. elif
4. Nested if

1. Simple if :- It is a type of Conditional Statement. It Can be used when a program has a single Condition & a single statement block.

* If the Condition is satisfied it executes the true statement block or else it stops the execution.

* Syntax :-
```
if Cond :
    True statement block.
```

* Flow diagram :-



Ex:-
```
age = int(input('enter a age:'))
if age >= 18:
    print('you are eligible to Vote')
```
→ enter a age : 23
you are eligible to Vote.

enter a age : 15.

① WAP to check whether the number is positive :-

→
```
num = int(input('enter a number :'))
if num >= 0:
    print(num, 'is a positive number')
```

→ enter a number : 34
34, is a positive number.

enter a number : −14

(2). WAP to check whether the number is even or not.

→ num = int(input('enter a num:'))
if num%2 == 0:
    print(num, 'is a even num')

→ enter a num : 42
42, is a even num.

enter a num : 51

(3) WAP to print the character present at even index of a string, if the length of the string is even.

→ String = input('enter a string :'))
if len(String)%2 == 0:
    print(String[::2], 'is present at even index of a string').

→ enter a String :- Saturday
Stra is present at even index of a string.

enter a String :- Welcome

enter a String :- 'Saturday'
'audy is present at even index of a String

(4) WAP to check whether a ~~str~~ character is vowel or not.

```
ch = input ('enter a character :')
if ch in 'AEIOUaeiou':
    print (ch, 'is a vowel')
```

→ enter a character : a
a is a vowel

enter a character : 5

Assignment :-

1. WAP to check whether the number is divisible by 5.
2. WAP to check whether the char is upper case or not.
3. WAP to check whether the string is keyword or not.
4. WAP to check whether two values are pointing to the same memory or not.

1.
```
num = int (input ('enter a number :'))
if n%5 == 0:
    print (num, 'is divisible by 5')
```

→ enter a number : 40
40, is divisible by 5

enter a number : 12

2. ch = input('enter a char:')
if ch.isupper():
print(ch,'is uppercase')

→ enter a char: U
U, is uppercase

enter a char: t

4. a = eval(input('enter a value:'))
b = eval(input('enter b value:'))
if a is b:
print('a & b are pointing to same Memory address')

→ enter a value: 22
enter b value: 22
a & b are pointing to same Memory address.

enter a value: 52
enter b value: 'hello'

2. ch = input('enter a char:')
if 'A'<=ch<='Z':
print(ch,'is uppercase')

→ enter a char: S
S, is uppercase

enter a char: l

3. import keyword
word = keyword.kwlist
S = input ('enter a string:')
if S in word:
print ('String is a keyword').

→ enter a String: While
String is a keyword
enter a string: OK

14/4/26

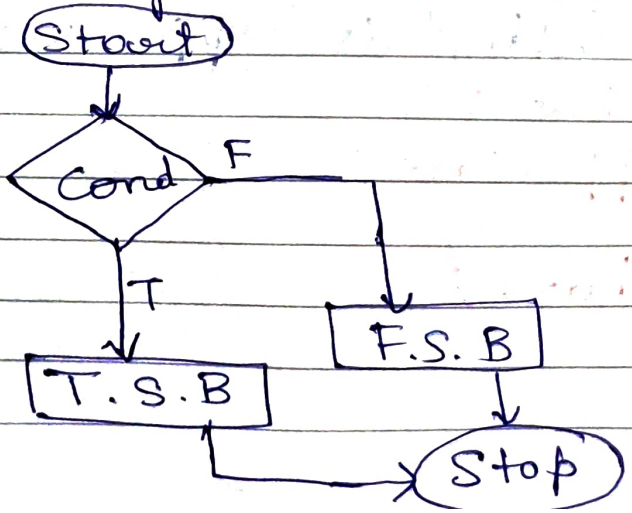
2. if-else :- It is a Conditional statement which is used when a single Condition has ~~Multip~~ Two Statement blocks.

* ~~If~~ the Condition is satisfied it executes True statement block. otherwise False statement block will be executed in else part.

* Syntax :-

```
if Cond :
    True Statement Block
else :
    False Statement Block
```

* Flow Diagram :-



① WAP to Check whether the String is palind-rome or not.

→
```
Str = input('enter a String:')
if Str == Str[::-1]:
    print('Palindrome')
else:
    print('not Palindrome')
```

→ enter a String: Mam
Palindrome

enter a String: Hello
not Palindrome

② WAP to print square of a number if it is even, print cube if it is odd.

→
```
num = eval(input('enter a number:'))
if num%2 == 0:
    print(num**2)
else:
    print(num**3)
```

→ enter a number: 14
196
enter a number: 15
3375

③ WAP to Check whether the value is Mutable or not.

→
```
val = eval(input('enter a value:'))
if type(val) in [list, set, dict]:
    print('Mutable')
else:
    print('iMMutable')
```

→ enter a value : 42
iMMutable.
enter a value : {42, 52, 62}
Mutable

Assignment :-

1. WAP to check whether the data is single value or not.
2. WAP to check whether the list Contains middle value or not.
3. WAP to check whether the char is special or not
4. WAP to check whether the given char is digit or not.
5. WAP to check whether the number is divisible by 9 or 3
6. WAP to check whether the number is Multiple of 3.

①
```
data = eval(input('enter a data:'))
if type(data) in [int, float, Complex, bool]:
    print("The data is single value")
else
    print("the data is not single value")
```

→ enter a data : 42.7
The data is single value
enter a data : [4,5,6]
The data is not single value.

5. num = int(input('enter a num :'))
if num%9==0 or num%3==0:
    print(num,'is divisible by 9 or 3')
else:
    print(num,'is not divisible by 9 or 3')

→ enter a num : 69
69, is divisible by 9 or 3
enter a num : 79
79, is not divisible by 9 or 3.

2. lis = eval(input('enter a list :'))
if len(lis)%2==1 :
    print("list Contain Middle value")
else :
    print("list doesn't Contain Middle value")

→ enter a list : [1,2,3,4,5]
list Contain Middle value
enter a list : [1,2,3,4,5,6]
list doesn't Contain Middle value.

3. ch = input('enter a char:')
if not('A'<=ch<='Z' or 'a'<=ch<='z' or '0'<=ch<='9'):
    print('It is special character')
else:
    print('not a special character')

→ enter a char: Z
not a special character
enter a char: $
It is special character.

4. num = input('enter a char:')
if '0'<= num <='9':
    print('It is a digit')
else:
    print('It is not a digit')

→ enter a char: 8
It is a digit:
enter a char: 89
It is not a digit.

6. num = int(input('enter a number:'))
if num%3 == 0:
    print('It is Multiple of 3')
else:
    print('It is not Multiple of 3')

→ enter a number: 36
It is multiple of 3
enter a number: 31
It is not Multiple of 3.

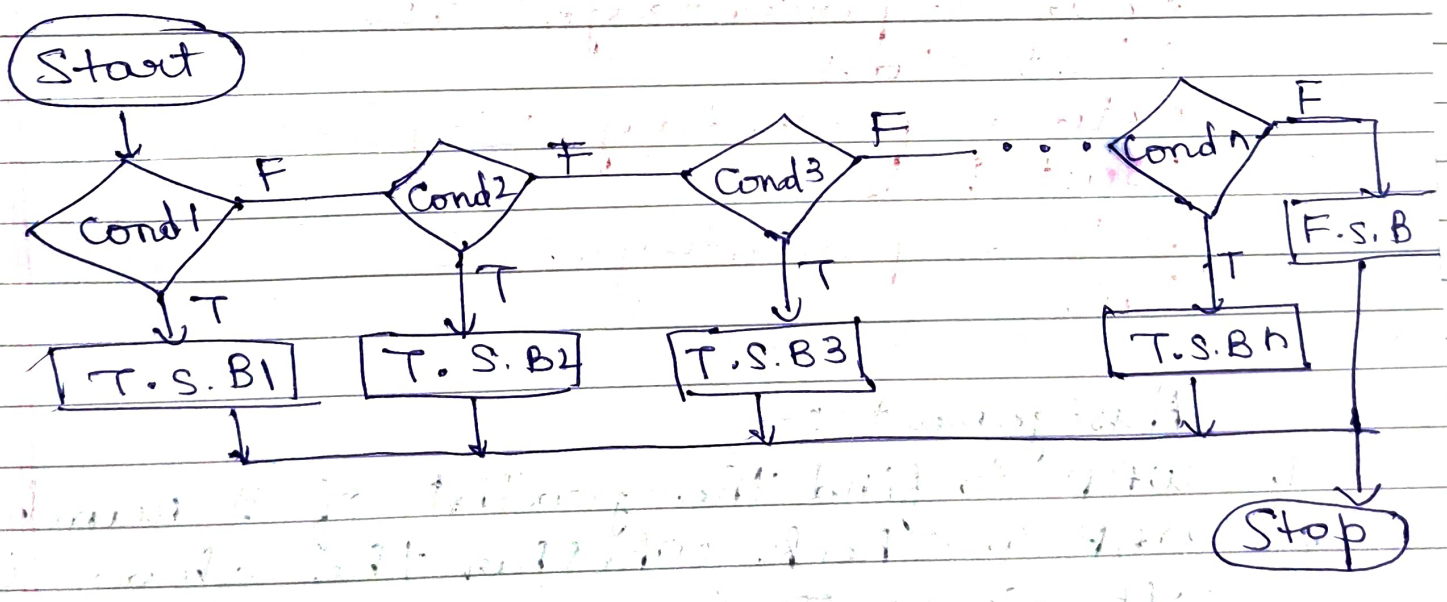
15/4/26

3. elif :- It is a Conditional statement which can be used whenever there is more than one condition.

* Syntax :-

```
if Cond1 :
        T.S.B1
elif Cond2 :
        T.S.B2
elif Cond3 :
        T.S.B3
  :
elif Cond n :
        T.S.Bn
else :
        F.S.B
```

* Flow chart :-



① WAP to check the marks of a student based on Percentage.

→
```
per = float(input('enter a percentage:'))
if per>100:
    print('invalid result')
elif per>=85 and per<=100:
    print('Distinction')
elif per>=60 and per<85:
    print('First class')
elif per>=50 and per<60:
    print('Second class')
elif per>=35 and per<50:
    print('Just Pass')
else:
    print('Fail')
```

→ enter a percentage: 84
First class
enter a percentage: 72
First class
enter a percentage: 37
Just Pass
enter a percentage: 25
Fail

Assignment :-

1. WAP to find the greatest of 3 numbers
2. WAP to check whether the char is uppercase or lowercase or digit or special char.

1.
```
num1 = int(input('enter num1:'))
num2 = int(input('enter num2:'))
num3 = int(input('enter num3:'))
if num1 > num2 and num1 > num2:
    print(num1, 'is greater')
elif num2 > num1 and num2 > num3:
    print(num2, 'is greater')
else:
    print(num3, 'is greater')
```

→ enter num1 : 64
enter num2 : 34
enter num3 : 22
64, is greater

enter num1 : 72
enter num2 : 80
enter num3 : 97
97, is greater

2.
```
ch = input('enter character:')
if 'A' <= ch <= 'Z':
    print(ch, 'is uppercase')
elif 'a' <= ch <= 'z':
    print(ch, 'is lowercase')
elif '0' <= ch <= '9':
    print(ch, 'is digit')
else:
    print(ch, 'is special character')
```

→ enter character : A
A, is uppercase
enter character : a
a, is lowercase
enter character : 7
7, is digit

enter character : $
$, is special character.

16/4/26.

① WAP to check whether the char is uppercase convert into lowercase, if char is lowercase convert into uppercase, if the char is digit print its ASCII value and if char is special then print it 5 times.

→
```
ch = input('enter a char:')
if 'A'<=ch<='Z':
    print('lowercase:', chr(ord(ch)+32))
elif 'a'<=ch<='z':
    print('uppercase:', chr(ord(ch)-32))
elif '0'<=ch<='9':
    print('ASCII value:', ord(ch))
else:
    print('Special character:', ch*5)
```

→
```
enter a char: m
uppercase: M
enter a char: M
lowercase: m
enter a char: 7
ASCII value: 55
enter a char: $
Special character: $$$$$
```

② WAP to check whether the integer is single digit, double digit or three digit or more than three digit.

→
```
num = input('enter a num:')
if len(num)==1:
```

```
    print('it is single digit')
elif len(num) == 2:
    print('it is double digit')
elif len(num) == 3:
    print('it is triple digit')
else:
    print('More then triple digit').
```

→
```
enter a num: 44
it is double digit
enter a num: 555
it is triple digit
enter a num: 1
it is single digit
enter a num: 6667
it is more then triple digit
```

(3). WAP to print 'Fizz' if the number is Multiple of three, print 'buzz' if the number is Multiple of 5 & print 'Fizzbuzz' if the number is multiple of both 3 & 5.

→
```
num= int(input('enter a num:'))
if num%3 == 0 and num%5 == 0:
    print('Fizzbuzz')
elif num%3 == 0:
    print('Fizz')
elif num%5 == 0:
    print('buzz')
else:
    print('not divisible')
```

→ enter a num : 42
Fizz
enter a num : 50
Buzz
enter a num : 15
FizzBuzz
enter a num : 43
not divisible.

17/4/26

(4) WAP to check the given points are lying in which quadrant.

```
→ x = int(input('enter a num :'))
  y = int(input('enter a num :'))
  if x>=0 and y>=0:
      print('1st Quadrant')
  elif x<0 and y>=0:
      print('2nd Quadrant')
  elif x<0 and y<0:
      print('3rd Quadrant')
  else:
      print('4th Quadrant')
```

→ enter a num : -11
enter a num : 12
2nd Quadrant.

```
→ x = int(input('enter x axis :'))
  y = int(input('enter y axis :'))
  if x==0 and y==0:
      print('origin')
```

```
elif x>0 and y>0:
    print('1st Quadrant')
elif x<0 and y>0:
    print('2nd Quadrant')
elif x<0 and y<0:
    print('3rd Quadrant')
elif x>0 and y<0:
    print('4th Quadrant')
elif x==0:
    print('on Y axis')
else:
    print('on X axis')
```

→
enter x axis: 0
enter y axis: 42
on Y axis

enter x axis: 60
enter y axis: 0
on X axis

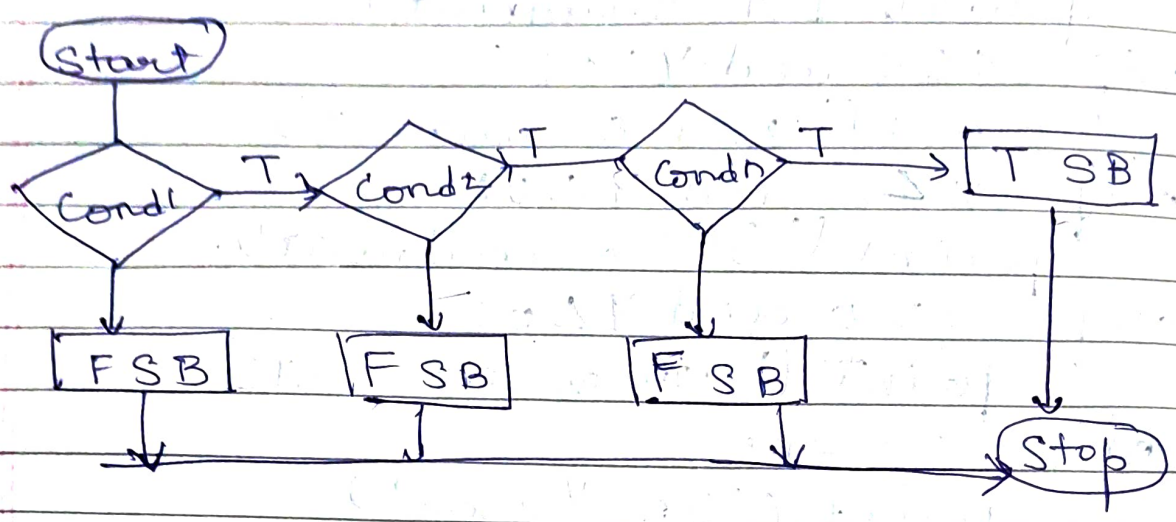
enter x axis: 0
enter y axis: 0
Origin.

4. Nested if :- If Condition inside another if Condition & it can be used when a Condition is depending on another Condition.

* Syntax:-

```
if Cond1:
    if Cond2:
        :
        if Cond n:
            TSB
        else: FSB
else else
FSB FSB
```

Flow Diagram :-



① WAP to check whether the character is Vowel or Consonant

→
```
ch = input ('enter a char:')
if 'A' <= ch <= 'Z' or 'a' <= ch <= 'z':
    if ch in 'AEIOUaeiou':
        print ('the char is Vowel')
    else:
        print ('the char is Consonent')
else:
    print ('the Char is not alphabet').
```

→
```
enter a char : $
the char is not alphabet
enter a char : S
the char is Consonant
enter a char : e
the char is Vowel
```

② WAP login to the instagram with valid username & password. (enter the

password only if the user name is valid).

→
```
user = input ("enter username:")
username = 'kavana'
password = 'kavu@2003'
if user == username:
    Psw = input (" enter Password:")
    if Psw == password:
        print ('Login Successful')
    else:
        print ('invalid Password')
else:
    print ('invalid username')
```

→ enter username: kavu
invalid username
enter username: Kavana
enter Password: kav@2003
invalid Password.
enter username: Kavana
enter Password: kavu@2003
Login Sucessfull.

③ WAP to print the Middle value of list only if it is string

→
```
lis = eval (input ('enter a list:'))
if len(lis)%2 == 1:
    m = lis[len(lis)//2]
    if m == str:
        print ('The middle value is:', m)
else:
```

(6). WAP to print the last value of list only if it is palindrome, str, starting with vowel

→
```
l = eval(input('enter a list:'))
last = l[-1]
if type(last) == str:
    if last == last[::-1]:
        if last[0] in 'AEIOUaeiou':
            print('The last value is:',
                  last)
        else:
            print('The last value
                  starting is not a vowel')
    else:
        print('The last value is
              not palindrome')
else:
    print('The last value is not a
          string')
```

→
```
enter a list : [4, 10, 15]
the last value is not a string.
enter a list : [4, 10, 15, 'OK']
the last value is not palindrome
enter a list : [15, 42, 'mam']
the last value starting is not a vowel
enter a list : [10, 20, 30, 'ana']
The last value is: ana
```

(ii) Looping Statement :- It is a types of Control statements which are use to execute the set of instructions repeatedly. There are 2 types :-

1. while loop
2. for loop.

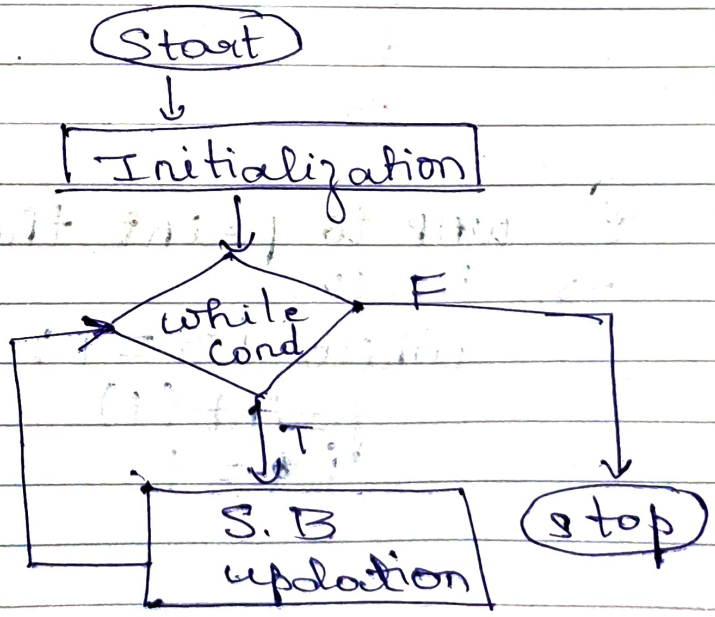
1) while loop :- It is a looping statement which executes the set of instructions repeatdly based on the condition.

* Here, initialization & updation of looping variable is mandatory.

* Syntax :-

```
initialization
while condition:
    S. B
    updation
```

* Flow chart :-



Ex:-
```
i = 1
while i <= 5:
    print ('Happy')
    i = i + 1
```
→
```
'Happy'
'Happy'
'Happy'
'Happy'
'Happy'
```

① WAP to print the numbers from 1 to 10.

→
```
i = 1
while i <= 10:
    print (i)
    i = i + 1
```
→
```
1
2
3
4
5
6
7
8
9
10
```

② WAP to print the numbers from 10 to 1

→
```
i = 10
while i >= 1:
    print (i)
    i -= 1
```
→
```
10
9
8
7
6
5
4
3
2
1
```

③. WAP to print the even number between 1 to 20.

→
```
i = 1
while i <= 20:
    if i % 2 == 0:
        print(i)
    i += 1
```

→ 2 4 6 8 10 12 14 16 18 20

④ WAP to print odd numbers blw 1 to 20

→
```
i = 1
while i <= 20:
    if i % 2 == 1:
        print(i)
    i += 1
```

→ 1 3 5 7 9 11 13 15 17 19

20/4/26

⑤ WAP to print the multiplication table for given number.

→
```
n = int(input('enter a num:'))
i = 1
while i <= 10:
    print(n, 'x', i, '=', n*i)
    i = i + 1
```

(or)

```
n = int(input('enter a num:'))
i = 1
while i <= 10:
    print(f'{n} x {i} = {n*i}')
    i = i + 1
```

→ enter a num: 2
2x1 = 2
2x2 = 4
2x3 = 6
2x4 = 8
2x5 = 10
2x6 = 12
2x7 = 14
2x8 = 16
2x9 = 18
2x10 = 20.

* f-String :- It is a formatted string literal which is used to insert the variable expression directly inside a string using {} braces.

(6) WAP to find the sum of n natural numbers.

```
→ n = int(input('enter a natural num:'))
  i = 1
  sum = 0
  while i <= n:
      sum = sum + i
      i += 1
  print(sum).
```

→ enter a natural num: 5
15

(7) write a program to find product of n natural number. or factorial number.

→
```
n = int(input('enter a num:'))
i = 1
fact = 1
while i <= n:
    fact = fact * i
    i += 1
print(fact).
```

→ enter a num: 5
120.

(8)
```
n = int(input('enter a num:'))
i = 1
j = n
while i <= n:
    print(i,j)
    i = i+1
    j = j-1
```

enter a num: 5
→ 1 5
2 4
3 3
4 2
5 1

(or)

```
i = 1
j = n
while j >= n:
    print(i,j)
    i = i+1
    j = j-1
```

(9) WAP to print Staect :-

```
n = int(input('enter a num:'))
i = 1
while i <= n:
    print(i * '*')
    i += 1
```

→ enter a num : 5

```
*
* *
* * *
* * * *
* * * * *
```

(10)

```
s = input('enter a string:')
i = 0
while i < len(s):
    print(s[i])
    i += 1
```

→ enter a string : Kavana

```
K
a
v
a
n
a
```

Assignment :-

(11) WAP to print the char present at even index of a str.

(12) WAP to extract Vowels present in str

21/4/26

(11) →
```
s = input("enter a string:")
i = 0
while i < len(s):
    print s[i]
    i += 2.
```
→ enter a string: Kavana
K
V
n

(12) →
```
s = input("enter a string:")
i = 0
vow = ''
while i < len(s):
    if s[i] in 'AEIOUaeiou':
        vow = vow + s[i]
    i += 1
print(vow)
```
→ enter a string: Kavana
aaa

(13) WAP to print factors of integer number.
→
```
n = int(input("enter a num:"))
i = 1
while i <= n:
    if n%i == 0:
        print(i)
    i += 1
```
→ enter a num: 15
1
3
5
15

(14) WAP to toogle a string.

→
```
S = input('enter a string:')
out = ' '
i = 0
while i < len(S):
    if 'A' <= S[i] <= 'Z':
        out = out + chr(ord(S[i]) + 32)
    elif 'a' <= S[i] <= 'z':
        out = out + chr(ord(S[i]) - 32)
    else:
        out = out + S[i]
    i = i + 1
print(out).
```

→ enter a string:- I aM Not HAppy@14
i Am nOT haPPY@14

(15) WAP to reverse a given number.

→
```
num = int(input('enter a num:'))
rev = 0
while num != 0:
    ld = num % 10
    rev = (rev * 10) + ld
    num = num // 10
print(rev).
```

→ enter a num: 4567
7654

Assignment :-

(16) WAP to extract all the even integers present in a tuple at odd index.

```
→ n = eval(input('enter a tuple:'))
  t = ()
  i = 1
  while i<len(n):
      if type(n[i]) == int:
          if n[i]%2 == 0:
              t = t + (n[i],)
      i += 2
  print(t)
```

→ enter a tuple :- (14, 92, 11, 16, 'bye', '52')
(92, 16, 52)

(17) WAP to find the sum of all the odd numbers blw the given range.

```
→ st = int(input('enter a num:'))
  ed = int(input('enter a num:'))
  Sum = 0
  while st<=ed:
      if st%2 == 1:
          Sum = Sum + st
      st+=1
  print(Sum).
```

→ enter a num: 10
enter a num: 20
75

(18) WAP to check whether the number is perfect or not.

→
```
n = int(input('enter a num:'))
Sum = 0
i = 1
while i < n:
    if n%i == 0:
        Sum += i
    i += 1
if Sum == n:
    print('Perfect number')
else:
    print('Not a perfect number')
```

→
```
enter a num: 6
Perfect number
enter a num: 8
Not a perfect number
```

(19) WAP to find the product of all the digits present in a number

→
```
num = int(input('enter a num:'))
product = 1
while num != 0:
    ld = num % 10
    prod = prod * ld
    num = num // 10
print(product)
```

→
```
enter a num: 123
6
```

(20) WAP to find the sum of individual digits of number.

```
n = int(input('enter a num:'))
Sum = 0
while n != 0:
    ld = n % 10
    Sum = Sum + ld
    n = n // 10
print(Sum)
```

→ enter a num: 456
15

(20) WAP to get the following output :-

```
S1 = '1 1 0 0 1 0 1 0'
S2 = '0 1 1 0 1 0 1 1'
i = 0
Count = 0
while i < len(S1):
    if S1[i] == S2[i]:
        Count = Count + 1
    i += 1
print(Count)
```

→ 5

(21) WAP to check the given number is prime or not.

```
n = int(input('enter a num:'))
i = 1
Count = 0
```

```
while i<=n:
    if n%i==0:
        count = count+1
    i+=1
if count == 2:
    print('The number is prime')
else:
    print('The number is not prime')
```

→
```
enter a num: 7
The number is prime
enter a num: 6
The number is not prime
```

23/4/26

(22) WAP to find the HCF of two numbers.

→
```
a = int(input('enter a num:'))
b = int(input('enter a num:'))
i = 1
hcf = 0
while i<=min(a,b):
    if a%i==0 and b%i==0
        hcf = i
    i+=1
print(hcf)
```

→
```
enter a num: 16
enter a num: 8
8
```

(23) WAP to Convert binary to decimal.

→
```
n = int(input('enter a binary value : '))
dec = 0
p = 0
while n != 0 :
    d = n % 10
    dec = dec + d * 2 ** p
    p = p + 1
    n = n // 10
print(dec)
```

→ enter a binary value : 1010
10

(24) WAP to Convert decimal to binary.

→
```
num = int(input('enter a num : '))
binary = ' '
while num != 0 :
    d = num % 2
    binary = str(d) + binary
    num = num // 2
print(binary)
```

→ enter a num : 80
1010000

(25) WAP to Count the number of words in String.

→
```
S = input('enter a string : ')
Count = 0        flag = False
i = 0
while i < len(S) :
    if S[i] != ' ' and Flag == False :
        Count += 1
        flag = True
```

```
elif S[i] == ' ':
    flag = False
i += 1
print (count).
```

→ enter a string: Hello this is Kavana
4

(26) WAP to check whether the number is palindrom or not.

```
n = int (input ('enter a num:'))
rev = 0
temp = n
while n > 0:
    rev = rev * 10 + n % 10
    n = n // 10
if temp == rev:
    print ('It is palindrome')
else:
    print ('It is not palindrome').
```

→ enter a num: 121
It is palindrome
enter a num: 4289
It is not palindrome

24/4/26

d) For loop:- for loop is used to iterate over sequence of values or collection
* It does not required any initialization or updation of looping variable.
* We can call for loop as self iterating loop because by default or

automatically the looping variable changes its value during each iteration without any updation.

* In for loop we can consider only collection data types like :- list, tuple, set, dict & string.
* By defaut its starts from beging of the collection & it executes until all the values from the collection is completed.
* Syntax:-

```
for item in collection:
    Statement Block
```

① WAP to print the character in a string

```
S = input ('enter a string :')
for i in S:
    print (i)
```

→ enter a string : Happy

H
a
p
p
y

② WAP to count the sum of all the integer present in a list.

```
S = eval (input ('enter a list :'))
Sum = 0
for i in S:
    if type (i) == int :
```

```
        Sum = Sum + i
    print ('The sum of list is:', Sum)
```

→ enter a list : [12, 23, 'hello', 'bye', 89]
The sum of list is ; 124

(3). WAP to Count the number of values present in a Collection without len()

```
C = eval (input ('enter a Collection:'))
Count = 0
for i in c:
    Count += 1
print (Count)
```

→ enter a Collection : [42, 52, 'hello', 'bye']
4

(4) WAP to reverse a string without using Slicing.

```
S = input ('enter a String :")
rev = ' '
for i in S:
    rev = i + rev
print (rev).
```

→ enter a String : Hello
olleH

(5) WAP to replace the space by present in a str by *.

```
S = input ('enter a String:')
out = ' '
```

```
for i in S:
    if i == " ":
        out = out + '*'
    else:
        out = out + i
print(out)
```

→ enter a String: I am Kavana
I * am * Kavana

25/4/26 Assignment:-

(6) WAP to extract all the non default values from a list.

(7) WAP to extract all the lowercase char in a str.

(8) WAP to remove duplicates from a list

(6).
```
lis = eval(input('enter a list:'))
non_default = [ ]
for i in lis:
    if bool(i) == True:
        non_default.append(i)
print(non_default)
```
→ enter a list: [42, 0, '', 32, 'hii']
[42, 32, 'hii']

(7).
```
S = input('enter a String:')
lower = ''
for i in S:
    if 'a' <= i <= 'z':
        lower = lower + i
print(lower)
```

→ enter a String :- I Am Kavana
m aana

⑧
```
lis = eval(input('enter a list:'))
dup = [ ]
for i in lis:
    if dup not in dup:
        dup.append(i)
print(dup)
```
→ enter a list : [12, 14, 'hii', 14, 'hii', 32]
[12, 14, 'hii', 32]

⑨ WAP to extract the vowels present at odd index of a String.

```
S = input('enter a String:')
Vowels = 'AEIOUaeiou'
extract = ' '
for i in range(0, len(S), 1):
    if i%2 == 1:
        if S[i] in Vowels:
            extract = extract + S[i]
print(extract)
```
→ enter a String : E Kavana
a a a

Range() :- It is an inbuit function used to generate sequence of number

* Syntax :-

range(SV, EV, Step)

* To get the o/p from range function it has to be type casted.
* whenever the starting value is 0 & step value is 1, we can ignore mention -ing them
* where ending value is always exclusive
* It can be used in for loop whenever it is required to work on index & number related programing.

(10) WAP to return the position of vowels present in the given string.

→
```
S = input ('enter a string:')
vow = 'AEIOUaeiou'
for i in range (0, len(S), 1):
    if S[i] in vow:
        print(i, end = ' ')
```

→ enter a string:- hello everyone
1 4 6 8 11 13

(11)
```
S = input ('enter a string:')
out = [ ]
for i in range (len(S)):
    out[i] = S[i]
print(out)
```

→ enter a string : Kavana
[0:'K', 1:'a', 2:'V', 3:'a', 4:'n', 5:'a']

27/4/26

(12) WAP to count the number of occurance of a specified ~~no~~ char.

```
num = input('enter a String:')
ch = input('enter a char:')
count = 0
for i in num:
    if i == ch:
        count += 1
print(count)
```

→ enter a String: Hello
enter a char: l
2

(13). WAP to extract upper, lower, digit, special present in a str to different o/p variables.

```
S = input('enter a String:')
lower = ''
upper = ''
digit = ''
Special = ''
for i in S:
    if 'a' <= i <= 'z':
        lower = lower + i
    elif 'A' <= i <= 'Z':
        upper += i
    elif '0' <= i <= '9':
        digit += i
    else:
        Speial += i
```

print ('the lowercase are :', lower)
print ('the uppercase are :', upper)
print ('the digits are :', digit)
print ('the special character are :', special)

→ enter a String : I aM KaVaNa @ 123
The lowercase are : aaaa
The uppercase are : IMKVN
The digits are : 123
The special character are :- @

(14). WAP to get following o/p :-
L = ['hai', 34, 3.4, 'hello', 90, 'byebye']
O = {'hai' : 'hi', 'hello' : 'ho', 'byebye' : 'be'}.

→ L = ['hai', : 'hi', 'hello' : 'ho', 'byebye' : 'be']
d = { }
for i in L :
    if type (i) == str :
        d[i] = i[0] + i[-1]
print (d)

→ {'hai', : 'hi', 'hello', : 'ho', 'byebye' : 'be'}

(15) WAP to get the following o/p using len function.
S = 'power Star'
out = {'power' : 5, 'Star' : 4}

→ S = input ('enter a string :')
d = { }
w = ' '
for i in S :
    if i != ' ' :

```
        w+=i
    else:
        d[w] = len(w)
        w=''
d[w] = len(w)
print(d)
```

using Split()

```
S='Power Star'
L=S.split()
d={}
for i in L:
    d[i]=len[i]
print(d)
```

→ enter a String: Power Star
{'Power': 5, 'Star': 4}

(6) WAP to Count the number of words in the String

```
S=input('enter a string:')
C=0
for i in S:
    if i==' ':
        C+=1
C+=1
print(C)
```

→ enter a String:- Happy birthday to u
4

* Split():- It is use to divide the String based on the required Character.
* By default it split the string by space

Syntax:-

```
Var.split()
```

Ex:- S='hii hello how are you'
S.split()

→ ['hii', 'hello', 'how', 'are', 'you']

* we can also split the string based on the required character.
* Syntax:-

var.split(ch) S='happybirthday')

Ex:- S.split('a')

→ ['h', 'ppy birthd', 'y']

* Syntax:- var.Split(ch, n)
* It will split the str based on the specified char & specified number of times.

Ex:- S='hii hello how are you'

S.Split(' ', 2)

→ ['hii', 'hello', 'how are you']

(17). WAP to Count the number of words in the string using split().

→ S = input('enter a string:')
P = S.split()
print(len(P))

→ enter a string: I am ok
3

28/4/26. (18) WAP to replace the special char present in a string by space.

→ S = input('enter a string:')
res = " "
for i in S:
    if ('A'<=i<='Z') or ('a'<=i<='Z') or ('0'<=i<='9'):

```
        res += i
    else:
        res += " "
print(res)
```

→ enter a String:- ABC@2003$14
ABC 2003 14

(19) WAP to get the following o/p:-
S = ['jiocinema.com', 'file.py', 'web.html', 'amazon.com', 'www.org']
out = ['com', 'py', 'html', 'org']

```
S = ['jiocinema.com', 'file.py', 'web.html', 'amazon.com', 'www.org']
out = []
for i in S:
    ex = i.split('.')[-1]
    if ex not in out:
        out.append(ex)
print(out)
```

→ ['com', 'py', 'html', 'org']

(20) S = ['jiocinema.com', 'file.py', 'web.html', 'amazon.com', 'www.org']
out = {'com': ['jiocinema', 'amazon'], 'py': ['file'], 'html': ['web'], 'org': ['www']}

```
d = {}
for i in S:
    j = i.split('.')
    if j[1] not in d:
```

```
        d[j[1]] = [j[0]]
    else:
        d[j[1]].append(j[0])
print(d)
```

→ {'com': ['jiocinema', 'amazon'], 'py': ['file'], 'html': ['web'], 'org': ['www']}

(21) d = ['jiocinema.com', 'file.py', 'web.html', 'amazon.com', 'www.org']

olp = {'jiocinema': 'com', 'file': 'py', 'web': 'html', 'amazon': 'com', 'www': 'org'}

→
```
d = {}
for i in S:
    j = i.split('.')
    d[j[0]] = j[1]
print(d)
```

→ {'jiocinema': 'com', 'file': 'py', 'web': 'html', 'amazon': 'com', 'www': 'org'}

(22) inp = 'abacbaacc'

out = {'a': 4, 'b': 2, 'a': 3}

→
```
inp = 'abacbaacc'
d = {}
for i in inp:
    if i not in d:
        l = inp.count(i)
        d[i] = l
print(d)
```

→ {'a': 4, 'b': 2, 'c': 3}

29/4/26

Intermediate Termination of loop.

* The process of stoping the loop before it finishes all its iterations when a Certain Condition occurs.
* Break :- It is a key word use to terminate the execution of the loop when break executes the loop stops Completely & Control will comes out of the loop.
* Once the break is executed further instructions inside the loop will not be executed.
* break keyword must be used only inside the loop

Ex:-
```
i = 1
while i <= 10:
    if i % 2 == 0:
        print(i)
        break
    i += 1
```
→ 2

```
for i in range(1,10):
    if i==3 or i==5:
        break
    print(i)
```
→ 1 2 3 4 5 6 7 8 9

Ex:-
```
for i in range(1, 20):
    print(i)
    break
```
→ 1

```
for i in range(1,10):
    break
print(i)
```
→ it doesn't print anything

(1) WAP to search an element inside the list (Linear search).

```
l = eval(input('enter a list:'))
k = eval(input('enter a key to search:'))
for i in range(0, len(l)):
    if l[i] == k:
        print('the key is found at', i, 'index')
        break
else:
    print('key not found')
```

→ enter a list: [10,20,30,40]
enter a key to search: 30
the key is found at 2 index

enter a list: [10,20,30,40]
enter a key to search: 50
key is not found

(2) WAP to check whether the list contains only positive numbers or not.

```
l = eval(input('enter a list:'))
c = 1
for i in l:
    if i <=0:
        c=0
        break
if c==1:
    print('positive number in a list')
```

```
    else:
        print ('only not positive numbers')
```

→ enter a list: [10, 20, 30]
positive numbers in the list

(3) WAP to check whether the list is homogenous or heterogenous.

→
```
l = eval (input ('enter a list:'))
for i in l:
    if type (l[0]) == type (i):
        print ('homogenous')
        break
    else:
        print ('heterogenous')
```

→ enter a list: [4, 5, 30]
Homogenous
enter a list: [4, 5.5, 6.0, 7]
heterogenous

* Infinite Loop :- It executes until the loop satisfied the condition or until it reaches break loop.
* It works by using while loop.

Ex:-
```
Import time
c = 0
while True:
    c += 1
    pwd = input ('enter a password:')
```

```
        if pwd == 'hii@123':
            print('Login Succesfully')
            break
        else:
            print('Invalid password try
                again!!')
            if c == 5:
                print('Too many attempts
                    wait for 10 sec')
                time.sleep(10)
                c = 0
```

→ enter a password : hii@123
Login succesfully
enter a password: kavaa
Invalid password try again!!
enter a password : kav@123
Invalid password try again!!
enter a password : kavana_gowda
Invalid password try again!!
enter a password : kav
Invalid password try again!!
enter a password :- kavu
Invalid password try again!!
Too many attempts wait for 10 sec.

enter a password : Hi@123
Login Sucasfully

30/4/16 ① WAP Check whether the number is prime are not

→
```
n = int(input("enter a num:"))
for i in range(2,n):
    if n%i==0:
        print("Not a prime number")
        break
else:
    print("Prime number")
```

→ enter a num: 8
Not a prime number
enter a num: 7
Prime number

② WAP to guess the number by using random function.

→
```
import random
n = random.randint(5,100)
while True:
    inp = int(input('enter a num:'))
    if inp==n:
        print('you guessed the num!!')
        break
    elif inp<n:
        print('enter a bigger num')
    else:
        print('enter a smaller num')
```

(3) WAP to find the first non repeating char in a string.

→
```
ch = input ('enter a string:')
for i in ch:
    if ch.count(i) == 1:
        print(i)
        break
```

→ enter a string: abcabccfg
e

* Continue:- Continue is a keyword which is use to skip the current iteration & move to the next iteration. It does not stop the loop even after executing the continue it goes back to the loop.
* Once the continue gets executed further instructions inside the loop will not be executed, directly the control goes back to the loop.

Ex:-
```
i = 1                          → 1
while i <= 10:                   3
    if i % 2 == 0:               5
        i += 1                   7
        continue                 9
    print(i)
    i += 1
```

Ex:- for i in range (1, 15):
    if i % 3 == 0 or i % 5 == 0:
        Continue
    print(i).

→ 1
2
4
7
8
11
13
14

2/5/26 Nested loop:- A loop inside another loop is known as nested loop.
Nested Collection:- A Collection inside another Collection is known as nested Collection.

* Syntax:-
for item1 in item:
    for item2 in item:

Ex:- for i in range (1, 5): (1, 2, 3, 4)
    for j in range (2, 10, 2): (2, 4, 6, 8)
        print (i, j)

→ 1 2 2 2 3 2 4 2
1 4 2 4 3 4 4 4
1 6 2 6 3 6 4 6
1 8 2 8 3 8 4 8

Ex:- for i in 'hello':
    for j in 'bye':
        print (i, j)

h b e b l b l b o b
h y e y l y l y o y
h e e e l e l e o e

Ex:- for i in 'hello':
    for j in 'bye':
        print(i+j)

→ hb eb lb lb ob
by ey ly ly oy
he ee le le oe

Ex:- S1 = 'abc' S2 = 'mno' S3 = 'xyz'.
for i in S1: (abc)
    for j in S2: (mno)
        for k in S3: (xyz)
            print(i+j+k)

→ amx amy amz
anx any anz } 1st iteration
aox aoy aoz

bmx bmy bmz
bnx bny bnz } 2nd iteration
box boy boz

cmx cmy cmz
cnx cny cnz } 3rd iteration
cox coy coz

Ex:- S = 'c Sala Cup namde'
d = {}
l = S.split()
for i in l: → c=0
    for j in i:
        c+=1
    d[i] = c

print(d)

→ {'e': 1, 'Sala': 4, 'Cup': 3, 'namde': 5}

Ex:- s = 'e Sala Cup namde'

```
d={}
l=s.split()
for i in l:
    vowel=''
    for j in i:
        if j in 'AEIOUaeiou':
            vowel+=j
    d[i]=vowel
print(d)
```

→ {'e': 'e', 'Sala': 'aa', 'Cup': 'u', 'namde': 'a'}

Ex:- inp = [123, 'bye', 456, 'good', 67.8, 9+8j]

out = [321, 'bye', 654, 'good', 67.8, 9+8j]

```
out=[]
for i in inp:
    if type(i)==int:
        rev=0
        while i>0:
            l=i%10
            rev=(rev*10)+l
            i=i//10
        out.append(rev)
    else:
        out.append(i)
print(out)
```

→ [321, 'bye', 654, 'good', 6.7.8, 9+8j]

5/5/26 ① WAP to Check whether the number is strong number or not.

→
```
n = int(input('enter a number'))
temp = n
Sum = 0
while n != 0:
    d = n % 10
    fact = 1
    for i in range(1, d+1):
        fact *= i
    Sum += fact
    n = n // 10
if Sum == temp:
    print('The given number {temp} is strong number')
else:
    print('The given number {temp} is not a strong number')
```

145
1! + 4! + 5!
1 + 24 + 120
145 = 145.

145 % 10 = 5

1, 5+1

[1, 2, 3, 4, 5]

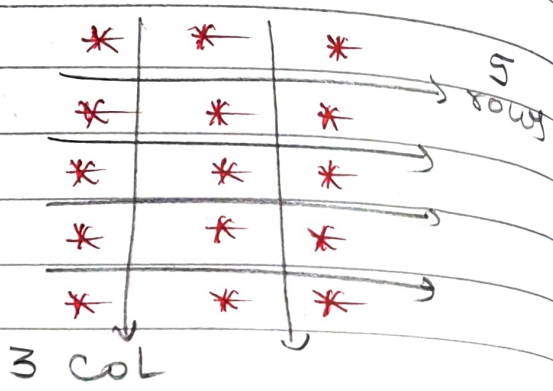
→ enter a number: 145
The given number 145 is strong number
enter a number: 12
The given number 12 is not strong number.

5/5/26 Pattern Matching:

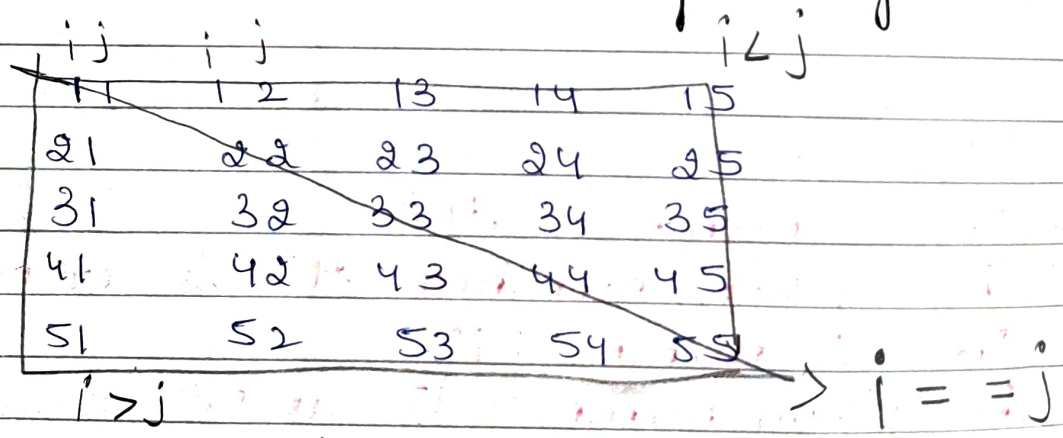
Pattern Programing :-

①
```
n = int(input('enter a num:')) row
s = int(input('enter a num:')) col
for i in range(1, n+1):          row
    for j in range(1, s+1):      col
        print('*', end=' ')
    print()
```

→ enter a num: 5
enter a num: 3



* Pattern with Primary diagonal.



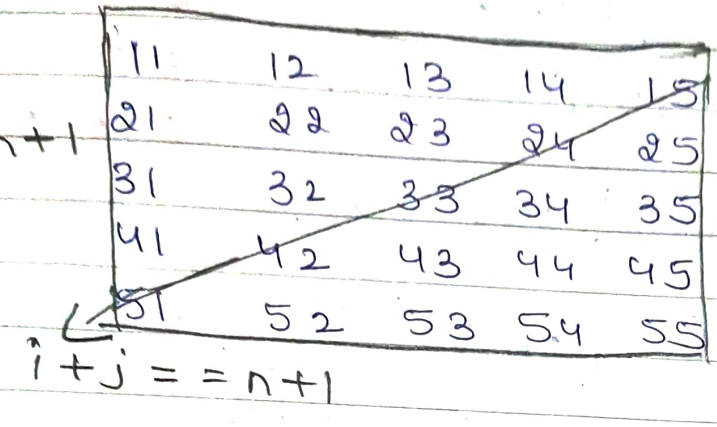
lower

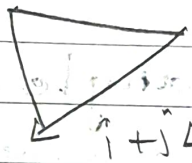
i==j
i>j
or
i>=j

upper

i<j
or
i<=j

* Pattern with secondary diagonal.





i+j>=n+1
lower triangle

i+j<n+1
or
i+j<=n+1
upper triangle

(1). #Square

```
for i in range(1,4):
    for j in range(1,4):
        print('*', end=' ')
    print()
```

* * *
* * *
* * *

(2). Primary Diagnol:-

```
for i in range(1,4):
    for j in range(1,4):
        if i==j:
            print('*', end=' ')
        else:
            print(' ', end=' ')
    print()
```

*
  *
    *

③. Right triangle: -

```
→ for i in range (1,4):
    for j in range (1,4):
      if i>=j:
        print ('*', end = '')
      else:
        print (' ', end = ' ')
    print().
```

④. Left triangle.

```
→ for i in range (1,4):
    for j in range (1,4):
      if i<=j:
        print ('*', end = '')
    print()
```

⑤ Secondary diag

```
→ n = int (input ("enter a num:"))
  for i in range (1, n+1):
    for j in range (1, n+1):
      if i+j == n+1:
        print ('*', end = ' ')
      else:
        print (' ', end = ' ')
  print ()
```

→ enter a num: 5

```
    *
  *
*
  *
    *
```

6/5/26.

(7)
```
n = int(input('enter a num:'))
for i in range(1, n+1):
    for j in range(1, n+1):
        if i>j:
            print(i, end=' ')
        else:
            print('', end=' ')
    print()
```

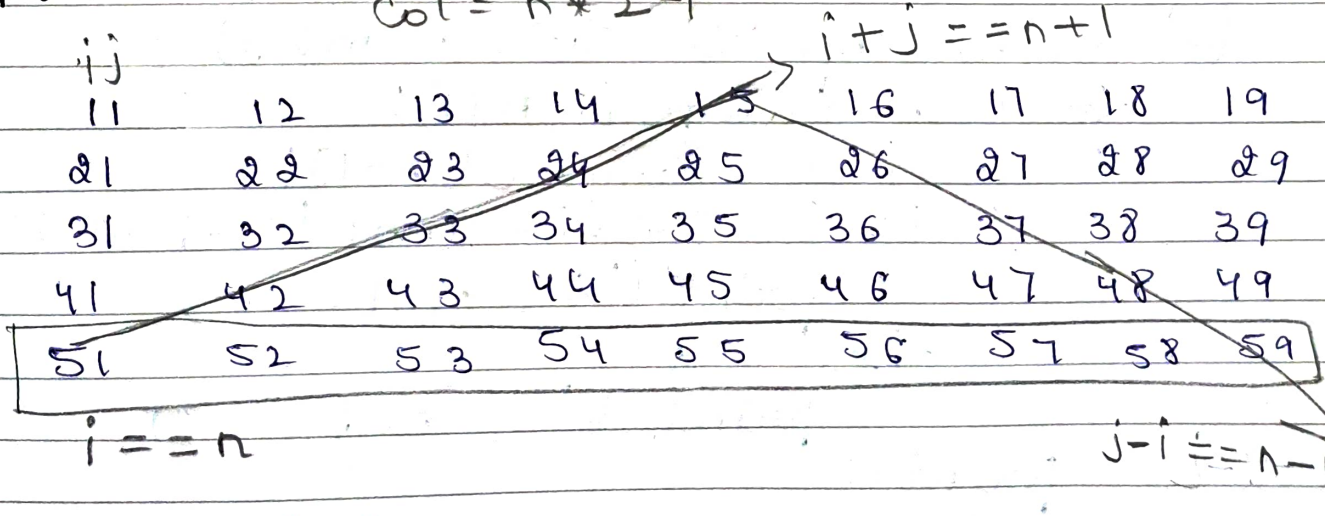
→
```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

(8)
```
n = int(input('enter a num:'))
for i in range(1, n+1):
    for j in range(1, n+1):
        if i>=j:
            if i%2==0:
                print(i, end=' ')
            else:
                print(j, end=' ')
    print()
```

→
```
1
2 2
1 2 3
4 4 4 4
1 2 3 4 5
```

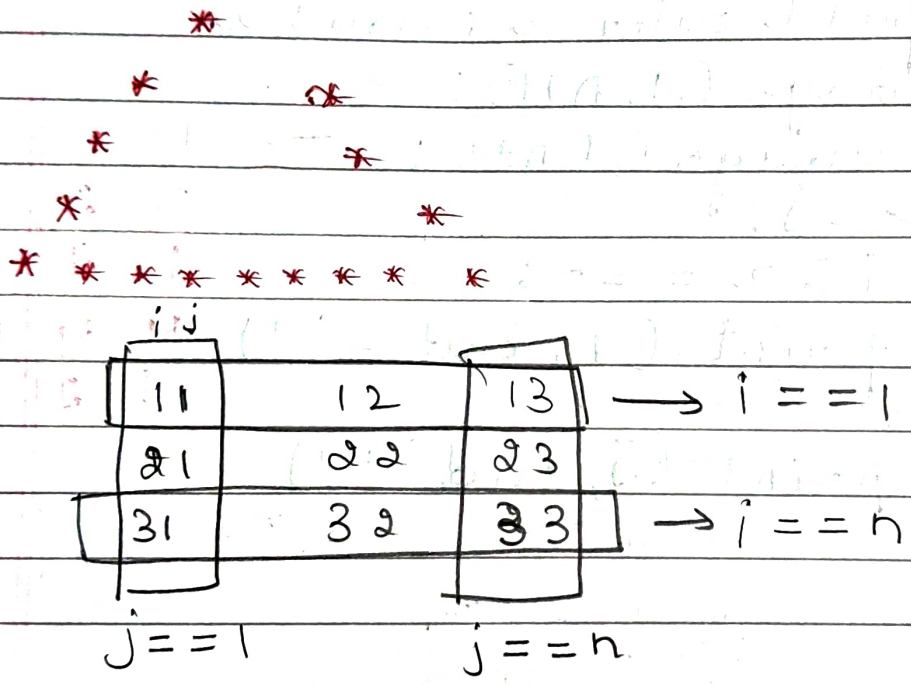
Pattern :-

row = 5
col = n*2-1



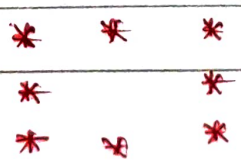
(9)
```
n = int(input('enter a num:'))
for i in range(1, n+1):
    for j in range(1, n*2):
        if i==n or i+j==n+1 or j-i==n-1:
            print('*', end=' ')
        else:
            print(' ', end=' ')
    print()
```

→ enter a num: 5



(10)
```
n = int(input('enter a num:'))
for i in range(1, n+1):
    for j in range(1, n+1):
        if i==1 or i==n or j==1 or j==n:
            print('*', end=' ')
        else:
            print(' ', end=' ')
    print()
```

→ enter a num: 3

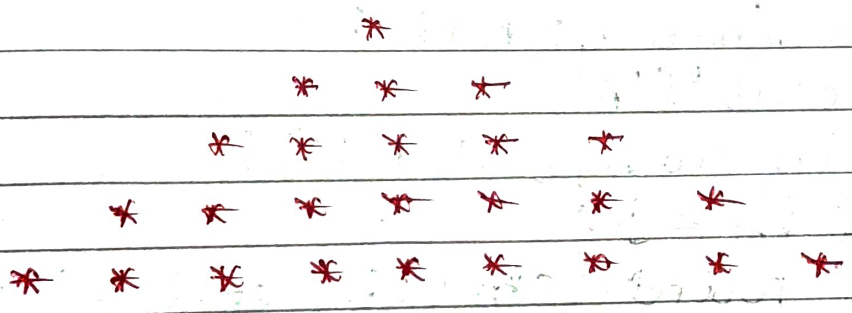


(11). n = int(input('enter a num:'))
for i in range(1, n+1):
    for j in range(n-i):
        print(' ', end=' ')
    for k in range(i*2-1):
        print('*', end=' ')
    print()

(or)

for i in range(1, n+1):
    print(' '*(n-i)+'*'*((i*2)-1))

→ enter a num: 5



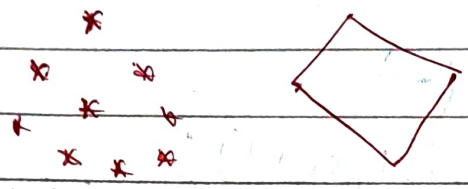
Assignment :-

row = 9
Col = 9

7/5/26.

(12)
```
num = int(input('enter a num:'))
for i in range(1,n+1):
    print(' '*(n-i)+'* '*(i))
for j in range(1,n):
    print(' '*(j)+'* '*(n-j))
```

→ enter a num:



(13)
```
n = int(input('enter a num:'))
for i in range(1,n+1):
    ch = ord('A')
    for j in range(1, n+1):
        if i>=j:
            print(chr(ch), end = ' ')
            ch+=1
        else:
            print(' ', end = ' ')
    print()
```

→ enter a num: 5

A
A B
A B C
A B C D

(14)
```
n=int(input('enter a num:'))
for i in range(1,n+1):
    for j in range(1,n+1):
        if i+j >=n+1:
```

```
            print(j, end = ' ')
        else:
            print(' ', end = ' ')
    ch = ord('A')
    for j in range(1, n+1):
        if i >= j:
            print(chr(ch), end = ' ')
        else:
            print(' ', end = ' ')
    print()

for i in range(1, n+1):
    for j in range(1, n+1):
        if i <= j:
            print(i, end = ' ')
        else:
            print(' ', end = ' ')
    ch = ord('A')
    for j in range(1, n+1):
        if i+j <= n+1:
            print(chr(ch), end = ' ')
            ch += 1
        else:
            print(' ', end = ' ')
    print()
```

→ enter a num : 5

```
        5 A
      4 5 A B
    3 4 5 A B C
  2 3 4 5 A B C D
1 2 3 4 5 A B C D E
1 1 1 1 1 A B C D E
  2 2 2 2 A B C D
    3 3 3 A B C
      4 4 A B
        5 A
```

(15)

```
n = int (input ('enter a num : '))
for i in range (1, n+1):
    for j in range (1, n+1):
        if (j == 1 and i > 1) or (j == n and
        i > 1) or (i == 1 and j > 1 and j < n) or
        (i == (n//2) + 1):
            print ('*', end = ' ')
        else:
            print (' ', end = ' ')
    print ()
```

→ enter a num : 5

```
* * *
*   *
* * * * *
*   *
*   *
```

A

8/5/26

* Functions :-
Functions are the names give to the memory block, where set of instructions are stored which are used to perform specific task.

* Types of functions :-
1. Built in functions
2. User-defined functions

* Utility functions :- These are the helper functions which are used to perform small & common task.
Ex:- len(), type(), max(), min(), sum().

* User defined functions :- These are the function which are created by the user to perform specific task based on there requirement.
* The function reduces the code repeatation
* we can define a function once & it can be called whenever it is needed.

Syntax :-

def fname (par1, par2 .... parn):
    S. B
    return

(par1, par2 .... parn) — formal arguments

fname (arg1, arg2 .... argn)

(arg1, arg2 .... argn) — actual arguments

def:- It is a keyword used to define a user defined function.

* The fname should follow the naming Conventions.
* Parameters the variables which are declared in the function definition they acts as place holders to receive the value when the function is called. Parameters are optional & it can also be refered as formal arguments.
* return:- is a keyword which is used to terminate the function & send the value back to the function call. Using return is not mandatory.
* Arguments:- are the actual values passed to a function when it is called. These values are assigned to the parameters.

Types of User defined function based on return & arguments:-

1. function without args & without return
2. function with args & without return
3. function without args & with return
4. function with args & with return.

1) Function without args & without return

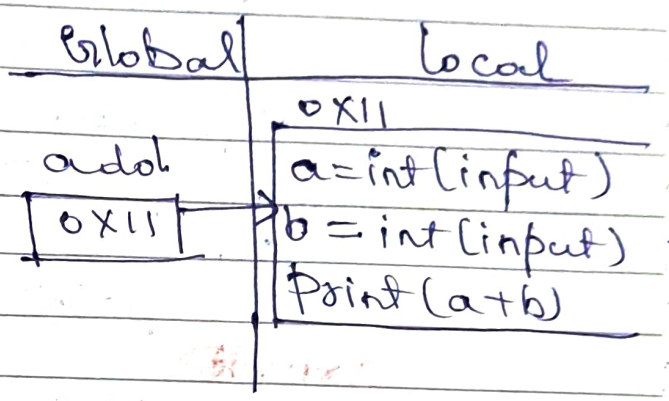
Syntax:-

```
def fname():
    [S.B]

fname()
```

Ex:- def add():
    a=int(input('enter a num:'))
    b=int(input('enter a num:'))
    print(a+b)
add()

→ enter a num: 10
enter a num: 20
30

add()
enter a num: 50
enter a num: 20
70



3/5/26

② Function with arguments & without return.

def fname(par1, par2 .... parn):
    S.B

fname(arg1, arg2 ... argn)

Ex:- def add(a,b):
    print(a+b)
add(10,20)

→ 30

Ex:- def factorial(n):
    fact=1
    for i in range(1,n+1):
        fact*=i
    print(fact)

→ factorial(5)
120

① whether the number is armstrong or not.

→
```
def arm(n):
    sum=0
    digit = len(str(n))
    for i in str(n):
        sum += int(i)**digit
    if sum == n:
        print('Armstrong number')
    else:
        print('not Armstrong number')
```

→ ~~arm(153)~~

arm(153)	arm(122)
Armstrong number	Not armstrong

②.
```
def max_char(s):
    d = {}
    for i in s:
        d[i] = d.get(i,0)+1
    max_ch = max(d, key = d.get)
    print(f'the max repeated char is {max_ch} and the count of it is {d[max_ch]}')
max_char('hello everyone')
```

3) without arguments and with return

```
def fname():
    S.B
    return v1, v2, ... vn
print (fname())
    or
u1, v2, ... vn = fname()
```

Ex:-
```
def sample():
    a = int(input('enter a num:'))
    b = int(input('enter a num:'))
    return a+b, a-b, a*b
print(sample())
        or
Sum, diff, prod = sample()
print(sample())(Sum, diff, prod)
```

→ enter a num: 2
enter a num: 3
(5, -1, 6) (or) 5 -1 6

14/5/26

① input = ['amazon.com', 'google.com', 'python.com py', 'web.html']
output = ['com', 'py', 'html']

→
```
def esc():
    l = eval(input('enter a list:'))
    Out = []
    for i in l:
        S = i.split('.')
        if S[1] not in out:
```

```
        out.append(s[i])
    return out
print(esc())
```

→ enter a list: ['amazon.com', 'google.com', 'python.py', 'web.html']
['com', 'py', 'html']

4) with Arguments and with return.

```
def fname(par1, par2 ... parn):
    S.B
    return v1, v2 ... vn
print(fname(arg1, arg2 .. argn))
```

(or)

```
var = fname(arg1, arg2 .... argn))
print(var)
```

Ex:-
```
def add(a,b):
    return a+b
print(add(10,20))
```

→ 30

① WAP to find the largest number in a list without using max().

→
```
def largest(l):
    large = l[0]
    for i in l:
        if i > large:
            large = i
    return large
print(largest(eval(input('enter a list:'))))
```

→ enter a list : [10, 40, 20, 60, 90, 12]
90

② WAP to find the second largest in a list.

→
```
def largest(l):
    large = 0
    second = 0
    for i in l:
        if i > large:
            second = large
            large = i
        elif i > second and i != large:
            second = i
    return second
print(largest(eval(input('enter a list:'))))
```
→ enter a list : [42, 15, 28, 37, 92, 52]
52

③ WAP to find the largest number in a list using max().

→
```
def largest(l):
    s = max(l)
    return s
print(largest(eval(input('enter a list:'))))
```
→ enter a list : [42, 32, 98, 71]
98

④. WAP to check whether the given number is strong number or not.

→
```
def strong_num(n):
    temp = n
    Sum = 0
    while n!=0:
        r = n%10
        fact = 1
        for i in range (1, r+1):
            fact *= i
        Sum = Sum + fact
        n = n//10
    if sum == temp:
        return 'A Number is strong num'
    else:
        return 'not a strong num'
print( strong_num(int(input('enter a num:'))))
```

→ enter a num: 145
A Number is strong num
enter a num: 23
not a strong num.

15/5/26

WAP to check whether the number is spy or not.

→
```
def prog(n):
    Sum = 0
    prod = 1
    while n!=0:
```

```
        d = n % 10
        Sum = Sum + d
        prod = prod * d
        n = n // 10
    if Sum == prod:
        print('Its a spy')
    else:
        print('its not a spy')
prog(int(input('enter a num:')))
```

→ enter a num: 123
Its a spy.
enter a num: 115
Its not a spy.

123 = 1+2+3 = = 1*2*3
= 6 = = 6

WAP to check whether the number is strong number or not :-

```
def factorial(n):
    fact = 1
    for i in range(1, n+1):
        fact = fact * i
    return fact

def strong(n):
    Sum = 0
    temp = n
    while n != 0:
        d = n % 10
        Sum = Sum + factorial(d)
        n = n // 10
    if Sum == temp:
        return True
    return False
```

WAP to print the strong number b/w 1 to 1000

→
```
def factorial(n):
    fact=1
    for i in range(1,n+1):
        fact=fact*i
    return fact

def strong(a):
    Sum=0
    temp=n
    while i!=0:
        d=n%10
        Sum=Sum+factorial(d)
        n=n//10
    return Sum==temp

def strong_num():
    for i in range(1,1001):
        if strong(i)==True:
            print(i)
strong_num()
```
func call

func call

→
```
1
2
145
```

16/5/26

Types of Arguments :-

1. Positional arguments
2. Default arguments
3. Keywords arguments
4. Variable length arguments

1) Positional arguments :- These are the arguments passed in the same order has parameters.

* It is mandatory to pass arguments to all the parameters.

Ex :- def student (name, phno, add):
print ('the student name &name&, phno is &phno& and the address is &add&')
Student ('Kavana', 8884551490, 'Mysore')

→ The student name Kavana, phno is 8884551490 and the address is 'Mysore.

2) Default arguments :- It is an argume-nt that takes the default value, if no value is provided during the function call.

* Parameters are assigned with a default value in the function declaration itself.

* Default parameters should be declar-ed after positional parameters.

Ex:- def Student (name, addr, phno = None, marks = 'does not exist'):
print (f' student name is {name}, phno is {phno} and addr is {addr} and marks is {marks}')
Student ('Kavana', 'Mysore')

→ Student name is Kavana, phno is none and addr is Mysore and marks is does not exist

3) Keyword Arguments:- Arguments which are passed using parameter names.

Ex:- def student (name, phno, addr):
print (f' the name of the student is {name}, phno is {phno} and address is {addr}')
Student (name = 'Kavana', addr = 'Mysore', phno = 4443338881)

→ The name of the student is Kavana, phno is 4443338881 and address is Mysore.

4) Variable length Arguments :- It is used when the number of arguments is unknown. There are 2 types:-

1. * args :- It accepts 0 to n number of positional arguments & creates a tuple.

Ex:- def Sam(*args):
    print(args)
Sam(1,2,3,4,5,6).

→ (1,2,3,4,5,6)

2. **kwargs:- It accepts 0 to n number of keword arguments & creates a dictionary.

Ex:- def sam(**kwargs):
    print(kwargs)
Sam(name='anu', phno='4446664510', addr='Mysore')

→ {'name':'anu', 'phno':4446664510, 'addr':'Mysore'}

Packing:- Packing is a process of Combining multiple values & storing them into a single Collection is known as packing. There are 2 types:-

1. Single/tuple packing:- Process of Combining 0 to n positional argument using *args into a tuple.
   * As it creates a tuple Collection it is also known as tuple packing.

2. Double/dictionary packing:- Process of Combining 0 to n keyword argument using **kwargs into a dict.
   * As it creates a dictionary Collection it is also known as dictionary packing

Ex:- def packing(*args, **kwargs):
    print('Single packing using *args:', args)
    print('Double packing using **kwargs:', kwargs)
packing()
→ Single packing using *args : ()
Double packing using **kwargs : {}
packing(1, 2, 24, 46)
Single packing using *args : (1, 2, 24, 46)
Double packing using **kwargs : {}
packing(1, 2, 24, 46, name='Kavana', age=23, addr='Mysore')
Single packing using *args : (1, 2, 24, 46)
Double packing using **kwargs :
{'name': 'Kavana', 'age': 23, 'addr': 'Mysore'}

Unpacking :- Process of extracting values from a collection into separate variables

Ex:- l = [1, 2, 34, 56, 5]
def sam(a, b, c, d, e):
    print(a, b, c, d, e, sep='\n')
sam(*l)
→ 1
2
34
56
5

(or)

a, *b = l.

→ a

1

b

[2, 34, 56, 5]

(or)

a, b, c, d, e = l

a

→ 1

b

→ 2

c

→ 34

d

→ 56

e

→ 5

18/5/26

Local variable :- These are the variable which are declared inside the function.

* we can access them only inside the function but not outside the function.
* They will get created when we call the function & after the function execution they will get destroyed.

Global variables :- which are declared outside function and it can be accessed anywhere in the program.

Scope of variables :- the scope of variable refers to the region of program where the variable can be accessed or used.

* Types of scope in python :-

1. **Local scope** :- The variable declared inside the function belongs to the local scope as it can be accessed only inside the function.

2. Enclosing scope / nested function scope :- when one function is inside another function the inner function can access variables of outer function.

Ex:- def outer():
    x = 100
    def inner():
        print(x)

3. Global scope :- A variable declared outside all the functions belongs to the global scope.

4. Built-in-scope :- Python provides pre-defined names or function which are available everyday.

Ex:- a = 10 } global variable
b = 10 }

def outer():
    c = 100 } enclosing
    d = 200 } variable
    print(a, b)
    def inner():
        x = 1 } Local variable
        y = 2 }
        print(x, y)
        print(c, d)
        print(a, b)

Note :- Python always searches for variable in LEGB order

Global keyword :- It is used to modify the global variables inside the function.

* when we try to modify global variable inside the function it will create a local variable & that modification will not affect the variable which is outside the function.

Ex :-
```
a = 100
b = 200                          → 1900
def loc():                         900   1000
    global a,b
    a=900
    b=1000
    print(a+b)
loc()
print(a,b)
```

Nonlocal Keyword: It is used in nested function to modify a variable from enclosing or outer function.

Ex :-
```
def Cam():
    x = 100                      → 200
    def Sam():
        nonlocal x
        x=200
        print(x)
```

Sam()
Cam()

19/5/26 ① a = 10
b = 20
def global_var():
global a
a = 100
print('inside func after Modi:', a, b)
print('outside func before Modi:', a, b)
global_var()
print('outside func after Modi:', a, b)

→ inside func after Modi: 100, 20
outside func before Modi: 10 20
outside func after Modi: 100 20

② def local():
global a, b
a = 100
b = 200
print(a, b)
local()
print('outside func:', a, b)

→ 100 200
outside function: 100 200

20/5/26.

① l = [10, 95, [1, 4, 18, [7, 16, 'bye', (1, 7, 9, {11, 12, 134, 'good'), 18, 99], 11, 17, 25, 100, [47, 16, 18]]

→
```
def sum_int(l):
    Sum = 0
    for i in l:
        if type(i) == int:
            sum += i
        elif type(i) in [list, tuple, set]:
            sum += sum_int(i)
    return sum
print(sum_int(l))
```
→ 1343

* Recursion :- It is a process where function calls itself repeatedly to solve the program by breaking it into smaller sub problem until termination condition or a base case is reached.

* Syntax :-

```
def fname(args):
    if termination_Condition:
        return values
    S.B
    return fname(args)
print(fname(values))
```

```
def fname(args):
    if termination_cond:
        return
    S.B
    fname(args)
fname(values).
```

① WAP to display factorial of a number:-

→
```
def fact(n):
    if n==1 or n==0:
        return 1
    return n * fact(n-1).
print(fact(5)).
```

→ 120

* getrecursionlimit():-

* Setrecursionlimit(500):- To increase blocks or decrease block.

②
```
import Sys
print('sys.getrecursionlimit())
Sys.setrecursionlimit(500).
def fact(n):
    if n==1 or n==0:
        return 1
    return n * fact(n-1)
print(fact(4999))
```

(3) WAP to print the n natural numbers:-

```
def natural(n):
    if n==1:
        return 1
    return n + natural(n-1).
print(natural(10)).
```

→ 55

(4).

```
def sam(n):
    if n==0:
        return
    print(n)
    sam(n-1)
sam(5).
```

→ 5 4 3 2 1

(5).

```
def sam(n):
    print(n, end=' ')
    if n==0:
        return
    sam(n-1)
    print(n, end=' ')
sam(5)
```

→ 5 4 3 2 1 0 1 2 3 4 5

22/5/26

(5) 5 4 3 2 1 0 1 2 3 4 5

```
def sam(n):
    print(n, end=' ')
    if n==0:
        return
    sam(n-1)
    print(n, end=' ')
sam(5)
```

~~→ 5 4 3 2 1 0 5 4~~ → 5 4 3 2 1 0 1 2 3 4 5

(6). Reverse a string :-

```
S = 'have fun'
rev = ''
i = 0
while i < len(S):
    rev = S[i] + rev
    i = i + 1
print(rev).
```

→ nuf evah

```
def fun(S, rev='', i=0):
    if i > len(S):
        return rev
    rev = S[i] + rev
    return fun(S, rev, i+1)
print(fun('hello'))
```

olleh

* Steps to Convert while loop program into recursion program.

1. Step 1 :- Initilization of all the looping variable should be done in function declaration.
2. Step 2 :- Write the termination Condition which is exactly opposite to the looping Condition.
3. Step 3 :- return the final o/p inside the termination Condition.
4. Step 4 :- Statement block should be written has it is inside the loop.
5. Step 5 :- updation of looping variable should be done in recursive function call.

(7) WAP to print the fibonacci Series.

Advantages of Recursion:-

1. Makes the code shorter & easier to understand the problems like trees, traversal, graph search algorithms

Disadvantage of Recursion:-

1. It uses more memory because each function is ~~Go~~ stored on the call stack.
2. Slower than normal loops.

25/5/26 Object Oriented Programming:-

OOPs is a programming paradim that uses classes & objects to organise the code & which helps to work on real time entities.

* In python every datatype is an in-built class & every variable which we create acts has object of that particular class.

Metaclass :- Metaclass is a class of class every inbuit class or User defined class are the objects of Metaclasses

* Metaclass present in python is type.

Ex:- a = 10

type(a)

→ <class 'int'>

type(int)

→ <class 'type'>

1) **Class** :- Class is a blueprint or a structure which used to store the properties & the functionalities of an object

* Properties can also be called as attributes which are nothing but Variable Created inside the class.
* Functionalities Can also be called has behaivour or Methods which are nothing but the functions inside the class.

2) Object :- Object is a instance of a class & it can access all the attributes & methods present inside the class.

* Syntax :-

```
class Cname :
↔   [ properties
      and
      functionality ]

object_name = Cname( arg1, arg2 ... argn )
                     ~~~~~~~~~~~~~~~~~~~~
                     not mandatory.
```

Ex :-
```
class School :
    Sname = 'Pentagon'
    Principle = 'Mytri'
    location = 'Vijayanagar'
S1 = School()
```
→ It gives Memory address of Class & object

* Syntax to accss the class properities :-
  Classname. attribute
* Syntax to accss the object properties:-
  Objname. attribute
* Syntax to modify the class property:-
  Cname. attribute_name = new_value
* Syntax to modify the object property:-
  objname. attribute_name = new_value

Ex:- class School:
    Sname = 'Anila'
    Princi = 'Mytri'
    location = 'Mysore'
S1 = School()
print (School. Sname, School. princi,
  School. location)
print (S1. Sname, S1. princi, S1. location).
→ Anila Mytri Mysore
  Anila Mytri Mysore
print ('After Modification')
School. Sname = 'Pentagon'
print (School. Sname, School. princi,
  School. location)
print (~~scho~~ S1. Sname, S1. princi, S1.
  location).
→ Pentagon Mytri Mysore
  Pentagon Mytri Mysore
  S1. location = 'Bangalore'
  print (School. Sname, School. princi,
    School. location)

print (S1.Sname, S1.princi, S1.location)

→ Pentagon Mytri Mysore
Pentagon Mytri Banglore

From the above example we can understand :-

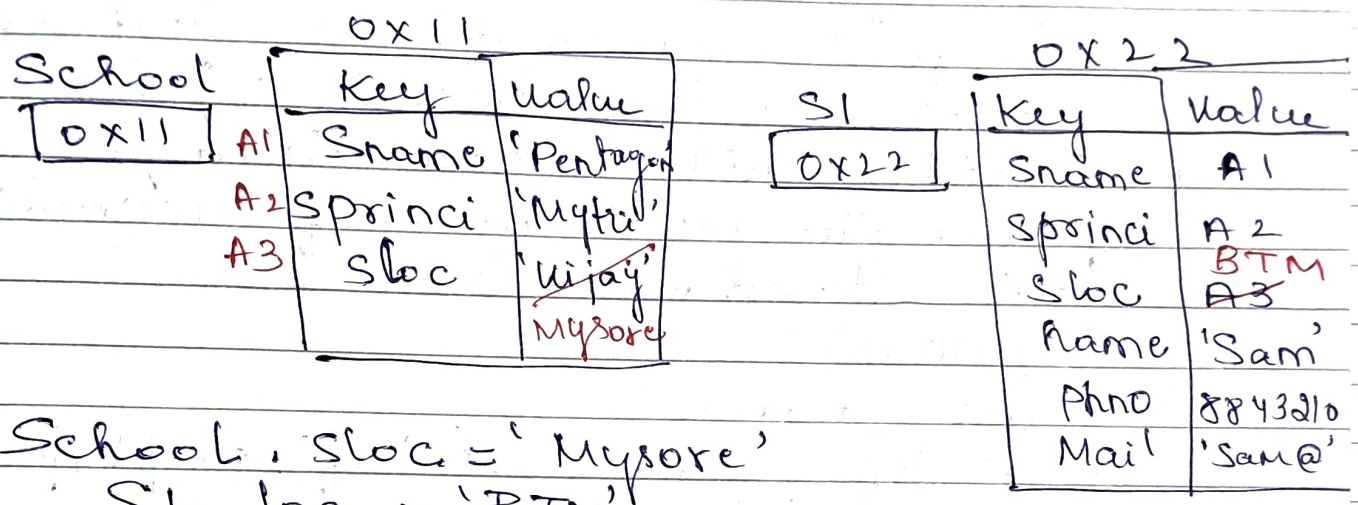
1. A class property can be accessed using both classname as well as objectname.
2. Modify the class attribute using Classname affects all the other objects that are using ~~that~~ the class attribute.
3. Modifying the a class attribute using an objectname updates only that particular object & doesn't affect the class attribute or the other objects.

Example of creating object property:-

```
Class School:
    Sname = 'Pentagon'
    SPrinci = 'Mytri'
    Sloc = 'Vijayanagar'
S1 = School()
S1.name = 'Sam'
S1.phno = '8843210678'
S1.mail = 'Sam@gmail.com'
print(S1.name, S1.phno, S1.mail)
S2 = School()
```

S2.name = 'Ram'
S2.phno = 8923467891
S2.mail = 'Sam@gmail.com'
print(S2.name, S2.phno, S2.mail)

→ Sam 8843210678 Sam@gmail.com
Ram 8923467891 Ram@gmail.com



School.Sloc = 'Mysore'
S1.loc = 'BTM'

* Inside the class all the attributes & methods are stored in the form of dictionary & we can access this dictionary using the special attribute called __dict__
* Syntax to access class dictionary:-
  Cname.__dict__
* Syntax to access object dictionary:-
  Objname.__dict__

26/5/26

Constructor :- Constructor is a method used to initialize the properties of an object.

* ___init___ :- It is a Constructor present in the python. It is a special method which is automatically called when an object is created. It takes the first parameter as Self and it refers to the Current object.
* Inside init method always the first parameter must be self according to the standards.

Syntax :-

```
Class Cname:
    def __init__(Self, prop1, prop2...):
        Self.prop1 = prop1
        Self.prop2 = prop2
        :
        Self.propn = propn
Obj = Cname(val1, val2 ... valn)
```

Ex :-

```
Class School:
    Sname = 'Pentagon'
    Sloc = 'Bangalore'
    def __init__(Self, name, phno, loc):
        Self.name = name
        Self.phno = phno
        Self.loc = loc
```

S1 = School ('Kavana', 9988665544, 'Mysore')
S2 = School ('Sam', 4432156789, 'Mysore')

Types of Methods :-
1. Instance / object Method
2. Class Method
3. Static Method

1) Instance / object Method :- This are the methods used to access & Modify the properties or attributes of an object

Imp

Syntax :-

* To display the object :-

```
def method_name (Self):
    print (Self.prop1, Self.pro2 ....)
obj_name.method_name().
```

* To display the obj using classname :-

```
def m_name (Self):
    print (Self.prop1, Self.prop2 ...)
~~obj_name.~~
classname.m_name (obj_name).
```

* To Modify the obj :-

```
def m_name (Self, new):
    Self.prop1 = new
obj_name.m_name (new_value)
```

* To Modify the obj using Classname :-

Imp
```
def m_name(self, new):
    self.prop = new
Classname.m_name(obj_name, new value)
```

Ex:-
```
class School:
    sname = 'pentagon'
    loc = 'Banglore'
    def __init__(self, name, phno, loc):
        self.name = name
        self.phno = phno
        self.loc = addr
    def display(self):
        print(self.name, self.phno,
              self.addr)
    def mod_phno(self, new):
        self.phno = new
    def mod_addr(self, new):
        self.addr = new
S1 = School('Kavana', 9988667891, 'Mysore')
S2 = School('Sam', 4832140891, 'Mandya')
S1.display()
S2.display()
```
→ Kavana 9988667891 Mysore
Sam 4832140891 Mandya

```
S1.mod_phno(4234561891)
S1.display()
```
→ Kavana 4234561891 Mysore

School.display(S1).
→ Kavana 9234561891 Mysore.

S2.Mod_addr('Banglore')
S2.display()
→ Sam 9832140891 Banglore

2) Class Method:- It is used to access & modify properties of a class.

* It takes cls as first parameter and cls refers to address of the class.
* To create a classmethod @classmethod is a decorator should be used otherwise the method will be considered as object method itself.

Syntax:-

* to access | display the class:-

@classmethod
def m_name(cls):
    print(cls.prop1, cls.prop2 ....)
Cname.M_name()

* To access | display the class by using object :-

@classmethod
def m_name(cls):
    print(cls.prop1, cls.prop2 ....)
Obj_name.m_name()

* To modify the class:-

```
@classmethod
def M_name (cls, new):
    cls.prop = new
Cname.M_name(value)
```

Ex:- ~~Class~~ School :

```
    Sname = 'Pentagon'
    loc = 'Vijayanagar'
    @classmethod
    def disp (cls):
        print (cls.Sname, cls.loc)
    @classmethod
    def Mod_loc (cls, new):
        cls.loc = new
S1 = School ('Kavana', 9988665510, 'Mysore')
S2 = School ('Ammu', 4418467891, 'Mandya')
School.disp()
School.mod_loc('BTM')
School.disp()
```

27/5/26

3) Static Method :- It is either a class method nor an object method. It just an utility or an helper function present inside the class.

Syntax :-

```
@staticmethod
def method_name ( ) :
    SB
```

Ex :-

```
class Student :
    Sname = 'Pentagon'
    Sloc = 'Bangalore'
    def __init__ (self, name, Addr):
        self.name = name
        self.addr = addr
    def display (self):
        self.static ()
    @staticmethod
    def static ():
        print ('the object are :', end = ' ')
        print (self.name, self.addr)
School.static ()

S1.display ()
```

→ The object are : Kavana Mysore.

Pillars of OOPS:-

1. Inheritance
2. Polymorphism
3. Encapsulation
4. Abstraction

1) Inheritance:- The process of deriving the properties from one class to another class is called as inheritance.

* It allows a class to reuse the attributes & methods of an existing class.
* Parent class:- It is a class from which we are deriving the properties. It is also called as super class / base class.
* Child class:- It is a class to which the properties are being derived. It is also called as derived class / sub class.
* Inheritance helps us to reduce the code redundancy & improve reusability.

Types of Inheritance:-

1. Single level
2. Multi Level
3. Multiple Level
4. Hierarchical level
5. Hybrid Level

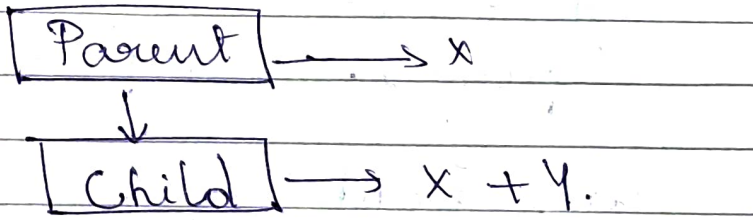
1) Single Level inheritance :- One child class inherits the properties & Methods from one parent class.

* Along with all the parent class properties child can have its own properties as well.

* Syntax :-

```
class parent :
    properties
    Behaviour
class child (Parent) :
    properties
    Behaviour
```

Diagram :-



Ex :-

```
class Parent :
    a = 10
    b = 20
class child (Parent) :
    x = 100
print (child.a, child.b, child.x)
```

→
10
20
100

28/5/26

Constructor chaining :- Process of invoking parent class constructor inside the child class constructor to avoid the re-initialization of properties which are already initialize in the parent class.

* When a child class contains the constructor the parent class constructor will not get executed we have to call it explicitly using Super function

Super function :- It is used to access parent class methods, variable and constructor inside a child class.

Syntax :-

Super().__init__(prop1, prop2 .... pron)

Ex :-

```
class Bank:
    bname = 'SBI'
    bloc = 'Mysore'
    def __init__(self, name, phno, bal):
        self.name = name
        self.phno = phno
        self.bal = bal
    def deposite(self):
        amt = int(input("enter a amount:"))
        self.bal = self.bal + amt
        print("the bal is:", self.bal)
    def withdraw(self):
        amt = int(input('enter a amt to
                        withdraw:'))
```

```
        if self.bal >= amt:
            Self.bal = Self.bal - amt
            print('the amount is withdraw, & the bal is :', Self.bal)
        else:
            print('insufficient bal, the bal is:', self.bal)
class bb(Bank):
    def __init__(Self, name, phno, bal, adhar, pan):
        Super().__init__(name, phno, bal)
        Self.adhar = adhar
        Self.pan = pan
    def check_bal(Self):
        print('the bal is :', Self.bal)
    def display(Self):
        print(Self.name, self.phno, Self.bal, Self.adhar, self.pan)
Cl = bb('Kavana', 8877618910, 8000, 82214287, 'GT11249')
Cl.display()
→ 'Kavana', 8877618910, 8000, 82214287, 'GT11249
Cl.withdraw()
→ enter a amt to withdraw : 5000
the amount is withdraw, & the bal is: 3000
Cl.deposite()
→ enter a amount : 2000
the bal is : 5000
```

Method chaining:- The process of calling Multiple methods in a single line on the same object

* Syntax:-

obj_name.mname1().mname2().mname3()..... mnamen()

* Where each & every method which we are calling in the same line should return self.

Ex:- class Sam:

```
    def method1(self):
        print('method 1')
        return self
    def method2(self):
        print('method 2')
        return self
    def method3(self):
        print('method 3')
        return self

ob = Sam()
ob.method1().method2().method3()
```

→ method1
method2
method3

30/5/26

Multiple Inheritance :- It is a process of deriving a properties from multiple parent classes to a single child class.

* Syntax :-

```
Class P1:
    properties
    behaiuar
Class P2:
    prop
    behav
Class P3:
    prop
    behav
Class Child (P1, P2, P3):
    prop
    behav
```

Ex:-

```
Class Batsman:
    def __init__ (Self, name, team):
        Self.name = name
        Self.team = team
    def bat (Self):
        print (Self.name, 'is batting')
Class Bowler:
    def __init__ (Self, name, team):
        self.name = name
        self.team = team
    def bowl (self):
        print (Self.name, 'is bowling')
```

```
class All_rounder (Batsman, Bowler):
    def feild (self):
        print ('feilding')
ob1 = All_rounder ('Krunal pandya', 'RCB')
ob2 = All_rounder ('Tim david', 'RCB')
ob1. bat()
```

→ Krunal pandya is batting.

```
ob2. bowl()
```

→ Tim david is bowling

(MRO) Method Resolution order :- It is a order in which python searches for methods & attributes in a class hirearchy.

* It is mainly used in Multiple Inheritance because here a single child class inherits the properties from Multiple parent classes. So, if ~~the both~~ all the parent classes has method or attributes with the same name child class will take such kind of properties from a class which we have mentioned first will inheriting. And it is because of MRO. which can also be called as C3 linearisation algorithm.

* Syntax :-

```
classname.__mro__
classname.mro
```

(or)

3) Hierarchical inheritance :- Process of deriving the properties from a single parent class to multiple child classes.

* syntax :-

```
class Parent:
    prop
    behav

class Child1(parent):
    prop
    behaiv

class Child2(parent):
    prop
    beh
    .
    .
    .

class childn(parent):
    prop
    beha
```

Ex:-

```
class Employee:
    Company = 'Google'
    CEO = 'Sundar Pichai'
    def __init__(self, name, id, DOJ, Sal):
        self.name = name
        self.id = id
        self.DOJ = DOJ
        self.Sal = Sal
    def display(self):
        print(f'ID : {self.id}')
        print(f'name : {self.name}')
        print(f'DOJ : {self.DOJ}')
```

```
        print(f'Sal : {self.sal}')
Class Developer(Employee):
    def skills(self):
        print('we work on python,
              html, CSS, ML')
    def designation(self):
        print(self.name, 'is a developer')
Class Tester(Employee):
    def designation(self):
        print(self.name, 'is tester')
obj = developer('Kavana', 1001, '21st june
      2026', 40000)
obj.display()
```

→ ID : 1001
Name : Kavana
DOJ : 21st June 2026
Sal : 40000

4) Hybrid Inheritance :- It is a combination of two or more types of inheritance.

Ex :-
```
Class Sampl:
    a=100
    b=12
Class samp2(Sampl):
    x=90
    y=80
Class samp3(Sampl):
    m=30
    n=50
```

```
class child (samp2, samp3):
    p = 500
ob = child()
print (ob.a, ob.b, ob.m, ob.n, ob.p,
    ob.y).
```

→ 100 12 30 50 500 80

5) Multi Level Inheritance :- The process of deriving the properties from one class to another class by Considering more than one level.

* And if forms chain of inheritance by inheriting the properties from derived class.

Syntax :-

```
class class1 :
    prop & Beha

class class2 (class1):
    prop & Beha

class class3 (class2):
    prop & Beha
    :
    :

class classn (class n-1):
    prop & beha
```

Ex:- class Resume:

```
    def __init__(self, name, phno, email):
        self.name = name
        self.phno = phno
        self.email = email
    def display(self):
        print(self.name, self.phno,
              self.email)
class Resume_1(Resume):
    def __init__(self, name, phno, email,
                 college, yop):
        super().__init__(name, phno,
                         email)
        self.college = college
        self.yop = yop
    def display(self):
        super().display()
        print('details of resume_1:',
              self.college, self.yop)
class Resume_2(Resume_1):
    def __init__(self, name, phno,
                 email, college, yop, domain):
        super().__init__(name, phno,
                         email, college, yop)
        self.domain = domain
    def disp(self):
        super().display()
        print('details of resume_2:',
              self.domain)
obj = display()
```

2) Polymorphism :- polymorphism is a ability of an method or operator to perform task depending on the object type.
There are 2 types :-
1) Compile time polymorphism
2) Run time polymorphism.

1) Compile time polymorphism :- It is also called as method overloading.
* It can be achieved by creating multiple methods with same name & with different parameters.
* While calling based on the parameter passed it invokes the respective methods.
* But python doesnot support method overloading because it is dynamically typed language.

2) Runtime polymorphism :-
1) Method overiding :- A child class provides the own implementation of a method that is already defined in parent class using same method name & parameter.

Ex:-
```
def Sam (a,b):
    print (a,b)
def Sam (a,b,c):
    print (a,b,c)
def Sam (a,b,c,d)
    print (a,b,c,d)
Sam (10,20)
```
→ error

Duck Typing :- It is a concept in python where the type of an object is less impact than the method & behaivor it provides.

* If it walks like a duck & quacks like a duck, Then it is a Duck

Ex:-
```
class Dog:
    def sound(self):
        print('bow bow')
class Cat:
    def sound(self):
        print('meow meow')
    def make_sound(animal):
        animal.sound()
d1 = Dog()
c1 = Cat()
make_sound(d1)   → bow bow
make_sound(c1)   → meow meow
```

Operator overloading :- operator overloading is the ability to change the behaivour of operators for user defined objects by implementing Dunder methods.

* As the operators are inbuilt they support only the objects of inbuilt but classes not the objects of user defined classes
* To use any operator an objects of user defined classes, the class should contain the respective magic methods.

Ex:- Class Arithemetic operators:

```
    def __init__(self, a):
        self.a = a
    def __add__(self, other):
        return self.a + other.a
    def __sub__(self, other):
        return self.a - other.a
    def __mul__(self, other):
        return self.a * other.a
    def __truediv__(self, other):
        return self.a / other.a
    def __floordiv__(self, other):
        return self.a // other.a
ob1 = Airthemeticoperators(10)
ob2 = Airthemeticoperator(20)
print(ob1+ob2)
```

→ 30 , -10 , 200 , 0.5 , 11

To access & modify values in list:-

```
Class Sample:
    def __init__(self, l):
        self.l = l
    def __getitem__(self, index):
        return self.l[index]
    def __setitem__(self, index, new):
        self.l[index] = new
    def __len__(self):
        return len(self.l)
ob = Sample([1, 2, 3, 4, 5])
print(ob[2])
ob[2] = 20.
```

→ 3

[1, 2, 20, 3, 4, 5]

9/6/26

Lambda function:- Lambda is an anonymous function which is used to solve basic programs in a single line.

* To create lambda function we don't use def keyword & it doesn't have name as well. So it is called as anonymous function.
* To create lambda function we should use lambda keyword & it should be stored in a variable. By calling that variable we can execute the lambda function.
* Syntax:-

1. Var = lambda args : Expression
2. Var = lambda args : TSB if Cond else FSB

Ex:- #Sum of two numbers:-

① Sum = lambda a,b : a+b
Sum (10,20).
→ 30

② #Even or odd.
Even_odd = lambda n : n%2 == 0
even_odd (42)
→ True
even_odd (11)
False

③ #WAP to print square of num if it is even or print Cube of a num if it is odd.

→ even_odd = lambda n : n**2 if n%2==0 else n**3.

even_odd(3)

→ 27

even_odd(2)

4

④ #WAP to create a lambda function which should return the sum of maximum 5 number and minimum 2 numbers.

→ num = lambda a, b, c=0, d=0, e=0 : a+b+c+d+e

num(10, 20)

→ 30

num(2, 3, 4, 5, 1)

15.

⑤ #WAP to check whether the number is palindrome or not.

→ palindrome = lambda S : S == S[::-1]

palindrome('OK')

→ False

palindrome('MAM').

→ True

11/6/26

Map() :- Map is an inbuilt function that applies given function to every item of an iterable & returns map object containing the transform values.

* Syntax:-

map(function, collection)

① #WAP to find the square of all the elements present in the list.

→ print (list(map(lambda n: n**2, eval(input("enter a list :")))))

② S = ['hello', 'good', 'morning']
S = [5, 4, 7]

→ S = ['hello', 'good', 'morning']
print(list(map(len, S)))

→ [5, 4, 7]

→ print(list(map(str.upper, S)))

→ ['HELLO', 'GOOD', 'MORNING']

③ l1 = [1, 2, 3, 4]
l2 = [5, 6, 7, 8]
out = [6, 8, 10, 12]

→ print(list(map(lambda a,b: a+b, l1, l2)))

→ [6, 8, 10, 12]

④ res = map(int, input("enter a str:"))
print(list(res)).

→ enter a str: 456789
[4, 5, 6, 7, 8, 9]

⑥. S = '123 22 43 12'
print(list(map(int, S.split())))
→ [123, 22, 43, 12]

Filter() :- filter() is used to extract the required elements from an iterable data based on the condition.

* It returns a filter object containing only the elements for which the condition is True.
* We have to type cast the filter object based on our requirements
* Syntax:-

var = filter(func, collection)
print(type(var)).

ex ● ① l = [1,2,3,4,5,6]
even = lambda n: n%2==0
res = filter(even, l)
print(list(res)).

(or)

print(list(filter(lambda n: n%2==0, eval(input('enter a list:'))))))
→ [2, 4, 6]

② WAP to filter positive number from a list.
→ print(list(filter(lambda n: n>=0, eval(input('enter a list:')))))
→ enter a list: [42, 62, -71, 82, -40]
[42, 62, 82]

(3). #WAP to filter string with length > 5.

```
print(list(filter(lambda s: len(s)>5,
    eval(input('enter a list :')))))
enter a list : ['Kavana', 'Geetha',
'RCB', 'Good']
['Kavana', 'Geetha']
```

(4). WAP to filter the str which starts with A

```
l = ['Apple', 'Mango', 'Ant', 'goa', 'AMMU']
print(list(filter(lambda l: l.
    startswith('A'), l)))
['Apple', 'Ant', 'AMMU']
```

12/6/26

(5). #WAP to square all the even numbers present in a list.

```
l = [12, 45, 3, 6, 7, 9, 14, 56, 78, 34]
even = lambda n : n%2 == 0
even_num = filter(even, l)
square = map(lambda n : n**2,
    even_num)
print(list(square))
```

(or)

```
print(list(map(lambda n : n**2,
    filter(lambda n : n%2 == 0, l))))
[144, 36, 196, 3136, 6084, 1156].
```

12/6/26

Comprehension :- Comprehension is a concise way of creating an output mutable collection in a single line of code using for loop.

Types of Comprehension :-

1. List Comprehension
2. Set Comprehension
3. dictionary Comprehension.

Syntax :- List Comprehension.

Imp

1. var = [expression for item in Collection]
2. var = [exp for item in coll if Condition]
3. var = [TSB if Cond else FSB for item in Collection]
4. var = [exp for item1 in coll for item2 in coll

1) List Comprehension :- It is a Concise & efficient way to create new list in a single line using for loop.

Ex :- ① WAP to create list of squares.

print [[i**2 for i in range (1,11)]]

→ [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

(2) #WAP to square all the even num b/w 1 to 10.

→ print([i**2 for i in range(1,11) if i%2==0])

→ [4, 16, 36, 64, 100]

(3) #WAP to square all the even numbers & Cube all the odd num.

→ print([i**2 if i%2==0 else i**3 for i in range(1,11)])

→ [1, 4, 27, 16, 125, 36, 343, 64, 729, 100].

(4) WAP to find the common elements b/w two list.

→ l1 = [42, 36, 47, 91]
l2 = [36, 91, 62, 71]
Com = []
for i in l1:
    if i in l2:
        Com.append(i)
print(Com).

(or)

print([i for i in l1 if i in l2])

→ [36, 91]

(5) WAP to flatten a nested list.

→ l = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flatten = []
for i in l:
    for j in i:

flatten.append(j)
print(flatten).

(or)

print([j for i in l for j in i])

→ [1, 2, 3, 4, 5, 6, 7, 8, 9]

(6). #WAP to remove ' ' from a list.

→ l = ['hi', ' ', 'ok', 'bye', ' ']
print([i for i in l if i != ' '])

→ ['hi', 'ok', 'bye']

(7). WAP to replace negative numbers with zero.

→ l = [42, -92, -40, 62, -19]
print([0 if i < 0 else i for i in l])

→ [42, 0, 0, 62, 0]

2) Set Comprehension :- It is a Concise & efficient way to create new set in a single line using for loop.

Syntax :-

1. Var = {Exp for item in Collection}
2. Var = {Exp for item in Coll if Cond}
3. Var = {TSB if Cond else FSB for item in Coll}
4. Var = {Exp for item1 in Coll1 for item2 in Coll2}

15/6/26

3) Dictionary Comprehension: Concise way of creating a dictionary in a single line using for loop.

Syntax :-

1. var = {k:v for item in collection}
2. var = {k:v for item in coll if Cond}
3. var = {k:v1 if Cond else v2 for item in coll}
4. var {k:v for item1 in coll1 for item2 in coll2}

(1) d = {}
for i in range (1,11):
    d[i] = i**2
print (d).

(or)

print ({i : i**2 for i in range (1,11)})
→ {1:1, 2:4, 3:9, 4:16, 5:25, 6:36, 7:49, 8:64, 9:81, 10:100}

(2) print ({i : i**2 if i%2==0 else i**3 for i in range (1,11)})
→ {1:1, 2:4, 3:27, 4:16, 5:125, 6:36, 7:343, 8:64, 9:729, 10:100}

③. l = {'hii', 35, 42.0, 'good', 'Man'}
print({i: i[0]+i[-1] for i in l if type(i) == str})
→ {'hii': 'hi', 'good': 'gd', 'Man': 'Mn'}

④ print({i: s[i] for i in range(len(s))})
s = 'string'.
→ {0: 's', 1: 't', 2: 'r', 3: 'i', 4: 'n', 5: 'g'}

* Enumerate function :- It is an inbuilt function which return both index & the value while iterating over the Collection.

⑤. print({i:j for i,j in enumerate(s)})
s = 'Come'
→ {0: 'C', 1: 'o', 2: 'M', 3: 'e'}

* Zip function :- It is used to iterate over multiple Collection simultaneously

⑥. l1 = [4, 5, 6, 7, 8]
l2 = ['ok', 'bye', 'Man']
print({i:j for i,j in zip(l1, l2)})
→ {4: 'ok', 5: 'bye', 6: 'Man'}

* Zip function stops iteration when the smallest Collection gets completed & it always returns the values of same positions from all the Collections

⑦ l1 = [1, 2.5, [3,4,5], 'a', (1,2,3)]
l2 = ['hii', 'bye', 'good', 'bad', 'Sad']
print({ i:j for i,j in zip(l1,l2)
if type(i) not in [list, dict, set]})

→ { 1: 'hii', 2.5: 'bye', 'a': 'bad',
(1,2,3): 'Sad' }

17/6/26

Decorators :- Decorator is a function which is used to add some additional functionalities or extra features to another existing function without Modying the original task

* Decorator is an higher order function because it takes another function as a parameter & it returns a function as output as well.

* There are 2 types of Decorators :-

1. In built decorator
2. User defined decorator.

1) In built decorator :- ~~These~~ These are the decorators which are created by the developers.

Ex:- @classmethod, @staticmethod, @abstractmethod etc.

2) User defined decorators :- These are the decorators which are created by the User based on their requirements.

* whenever the pre-task & the post-task is same for multiple functions we can use decorator to perform them without Modifying the main task of the function.

Syntax :-

1. def dector_name(func):
    def wrapper(*args, **kwargs):
        pre task
        func(*args, **kwargs)
        post task
    return wrapper.

2. @decorator_name
   def main_func(par1, par2 ....):
   S. B

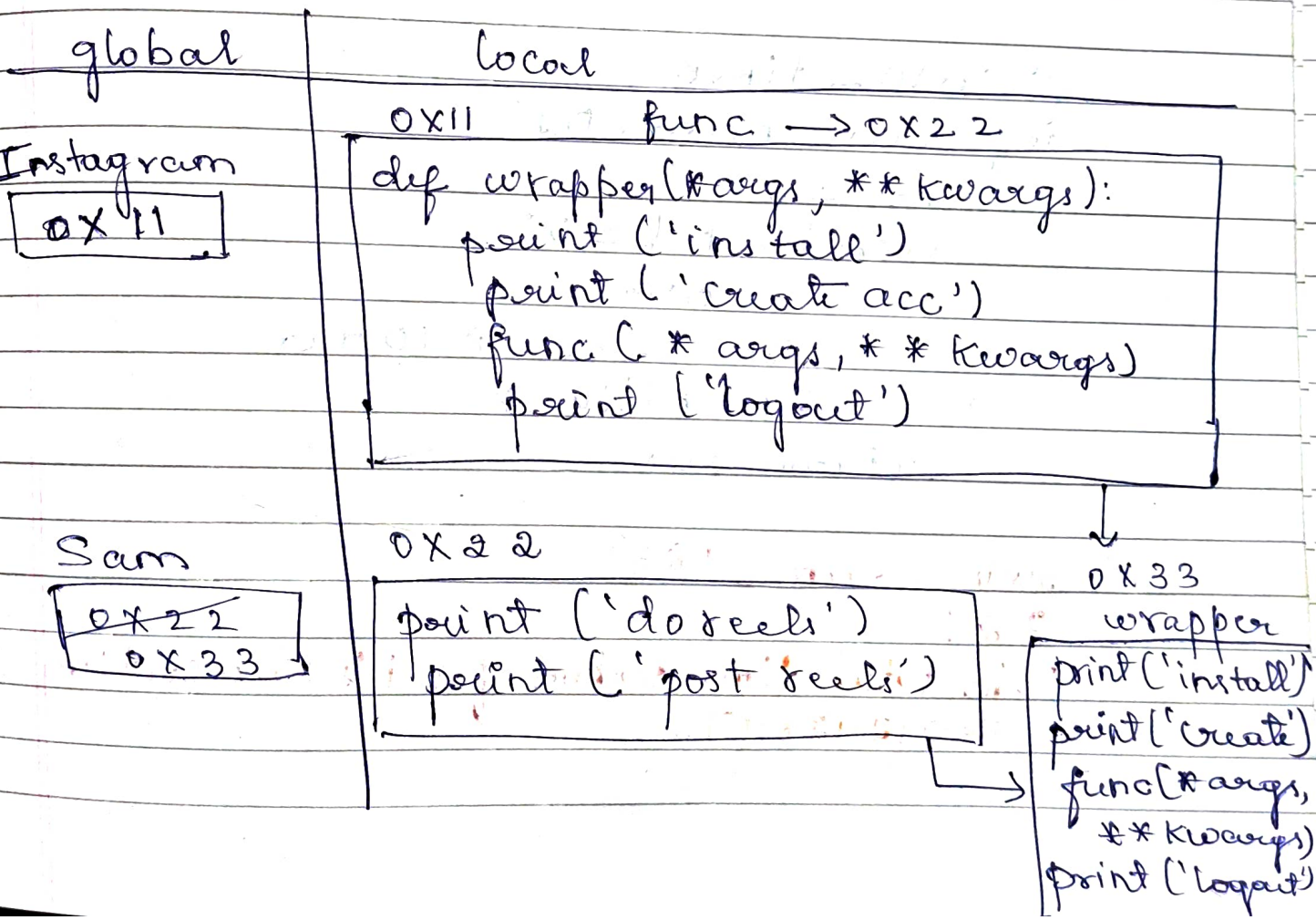
Ex:- def instagram(func):
    def wrapper(*args, **kwargs):
        print('install the app')
        print('create the account')
        print('update the account')
        func(*args, **kwargs)
        print('logout')
    return wrapper.

@instagram
def sam():
    print('do reels')
    print('post the reels')

@instagram
def jam():
    print('Chatting').

→ sam()
install the app
create the account
update the account
do reels
post the reels
logout.

jam()
install the app
Create the account
Update the account
Chatting
Logout



① Create the decorator to cal the execution time of a program.

→

```
import time
def cal_time(func):
    def wrapper(*args, **kwargs):
        st = time.time()
        rev = func(*args, **kwargs)
        print(rev)
        et = time.time()
        print('the total time taken for its execution is :',
              et - st)
    return wrapper

@calc_time
def rev_num(n):
    rev = 0
    while n != 0:
        d = n % 10
        rev = rev * 10 + d
        n = n // 10
    return rev
```

→

```
rev_num(123)
321
The total time taken for its execution is
0.07242897 3
```

16/26

(2). Create a decorator which gives 10 sec of time delay before executing any program or function.

→
```
import time
def delay_time(func):
    def wrapper(*args, **kwargs):
        st = time.sleep(10)
        rev = func(*args, **kwargs)
        print(rev)
        print('10 sec of time delay
            before executing the program', rev)
    return wrapper

@delay_time
def rev_num(n):
    rev = 0
    while n != 0:
        d = n%10
        rev = rev * 10 + d
        n = n//10
    return rev.
```

→ rev_num(4567)

10 sec of time delay before executing the program : 7654